CHAPTER 1: INTRODUCTION

1.1 Overview of the Project

The Smart Campus system is a comprehensive, web-based academic management platform developed using Python and the Flask micro web framework. The system integrates a MySQL relational database, a Natural Language Processing (NLP) module powered by the TextBlob library, and a rule-based predictive analytics engine to deliver a unified, data-driven experience for three distinct user roles: Students, Faculty Members, and Administrators. By combining performance monitoring, automated prediction, and sentiment-based feedback analysis within a single accessible web application, Smart Campus transforms the way educational institutions manage and respond to student academic data.

In most college environments today, student performance data — attendance records, examination marks, and faculty feedback — is maintained in isolated systems that rarely communicate with each other. This fragmentation makes real-time analysis difficult, delays the identification of at-risk students, and results in missed opportunities for timely academic intervention. The Smart Campus platform directly addresses these challenges by consolidating all relevant data into one MySQL database and presenting role-specific insights through dedicated dashboards.

Students log in to view their marks in three core subjects — Database Management Systems (DBMS), Python Programming, and Artificial Intelligence (AI) — along with their attendance percentage, computed overall average, and three AI-generated outputs: a failure risk classification (High Risk, Moderate Risk, or Low Risk), a placement readiness score (High, Medium, or Low Placement Chance), and a personalized study suggestion. Faculty members access a student list and can view or update their own professional profiles. Administrators have full control over student records and can view sentiment-labeled feedback submitted by any student.

A key innovation of the system is the TextBlob-powered sentiment analysis module. Every time a student submits textual feedback, the system automatically computes its polarity and classifies it as Positive, Neutral, or Negative — eliminating the need for administrators to manually read and interpret every submission. The prediction engine in model.py uses

transparent, rule-based logic to generate instantly understandable academic insights. Together, these features make Smart Campus a genuinely intelligent, yet fully interpretable, academic management solution.

Developed entirely on open-source technologies and deployable on standard hardware without any software licensing costs, the Smart Campus system is both accessible and scalable. Its modular Flask architecture is designed to accommodate future enhancements including machine learning-based prediction models, mobile application interfaces, cloud deployment, and real-time notification systems.

1.2 Background and Motivation

The smart campus concept is an extension of the broader smart city movement, applying the principles of digitization, data-driven decision-making, and intelligent automation to the educational environment. While large research universities and international institutions have invested heavily in Learning Management Systems (LMS) and data analytics platforms, the undergraduate colleges of developing nations — particularly in India — continue to rely predominantly on paper-based processes or rudimentary spreadsheet-based record-keeping.

The primary motivation for developing the Smart Campus system is the recognition that students who are at risk of academic failure rarely receive timely, actionable guidance. In a conventional college setting, attendance may fall below acceptable levels for weeks before any faculty member notices, and a student struggling across multiple subjects may not be flagged until examination results are released — by which time meaningful intervention is no longer possible. A system that continuously computes and displays performance metrics and proactive suggestions at every student login can make a measurable difference in academic outcomes.

The project is also motivated by the inefficiency of traditional feedback collection mechanisms. Student feedback forms are distributed and collected at the end of each semester, but the sheer volume of responses makes systematic analysis impractical. As a result, valuable insights from students about teaching quality, campus facilities, and academic support are often ignored. By automating feedback sentiment classification with TextBlob, Smart Campus ensures that the overall mood of the student body is always visible to administrators, enabling timely and informed institutional responses.

Additionally, the project is motivated by the desire to demonstrate that genuinely intelligent web applications can be built using freely available, well-documented Python libraries without requiring specialized data science infrastructure. The accessibility of tools like Flask, TextBlob, and mysql-connector-python makes it feasible for undergraduate college projects to incorporate real AI functionality — bridging the gap between academic learning and industry-relevant application development.

1.3 Problem Statement

The following problems in traditional academic management motivate the development of the Smart Campus system:

- Fragmented Data: Marks, attendance, and feedback are stored in separate, disconnected systems with no unified view of a student's overall academic health.
- Reactive Monitoring: At-risk students are identified only after academic failure has occurred, leaving no time for effective remediation or support.
- No Personalized Guidance: All students receive the same generic advice regardless of their individual performance profiles.
- Wasted Feedback Data: Student feedback is collected but rarely analyzed systematically due to the volume of submissions.
- Absence of Role-Based Access: Legacy systems often lack proper access controls, creating data privacy risks and workflow inefficiencies.
- High Administrative Overhead: Manual data entry and record-keeping is time-consuming and error-prone, leading to inaccurate academic records.

The Smart Campus system addresses all of these problems by providing a unified, intelligent, and accessible platform with automated analytics, NLP-based feedback classification, and role-based dashboards.

1.4 Objectives of the Project

1. Develop a secure, role-based authentication system providing customized dashboards for Student, Faculty, and Administrator user types.
2. Implement a student academic performance tracking module that consolidates subject-wise marks (DBMS, Python, AI) and attendance for every registered student.
3. Build a rule-based predictive analytics engine computing failure risk, placement readiness score, and a personalized study suggestion for each student.

4. Integrate an NLP-based sentiment analysis module using TextBlob to automatically classify student feedback as Positive, Neutral, or Negative.
5. Create an administrative interface enabling viewing, selection, and updating of student academic records, and review of sentiment-labeled feedback.
6. Develop a faculty module allowing faculty to view the student list and manage their own professional profile (view and update).
7. Design a normalized MySQL database storing all system data with proper relational constraints and referential integrity.
8. Build a user-friendly web interface using HTML, CSS, and Jinja2 templates that is intuitive for users of all technical backgrounds.
9. Ensure data security through parameterized SQL queries throughout the application to prevent SQL injection vulnerabilities.
10. Create a modular, well-documented codebase that can be easily extended with additional features in future versions.

1.5 Scope of the Project

1.5.1 Inclusions

- Web-based application accessible through any standard browser on any networked device, with no client-side software installation required.
- Role-based authentication for Student, Faculty, and Admin with role-appropriate dashboards.
- Student dashboard with marks, attendance, overall percentage, failure risk, placement score, and study suggestion.
- Faculty dashboard with student list view and a faculty profile management module.
- Admin dashboard supporting full student record viewing, updating, and feedback sentiment review.
- Automated TextBlob NLP sentiment analysis of student feedback (Positive / Neutral / Negative).
- MySQL relational database with four tables: users, students, faculty, and feedback.
- Flask routing and Jinja2 template rendering for all web pages.

1.5.2 Exclusions

- No native mobile application; the system is web-browser only.

- No advanced ML algorithms (neural networks, random forests); prediction logic is rule-based.
- No real-time push notifications or email alerts.
- No timetable, fee payment, library, or assignment management modules.
- No multi-institution cloud deployment in the current version.

1.6 Organization of the Report

Chapter 2 (Literature Survey) reviews existing academic management systems, AI-based student performance prediction research, and NLP sentiment analysis in educational contexts, identifying gaps that the Smart Campus system addresses.

Chapter 3 (System Analysis) covers feasibility analysis (technical, operational, economic), functional and non-functional requirements, and hardware and software specifications.

Chapter 4 (System Design) details the system architecture, DFDs, use case and ER diagrams, database table structures, module design, and sequence diagrams.

Chapter 5 (Technology Stack) explains all technologies and tools — Python, Flask, MySQL, TextBlob, Jinja2 — with details on how each is used in the project.

Chapter 6 (Implementation) describes the implementation of every module with code explanations, image placeholders for interface screenshots, and algorithm descriptions.

Chapter 7 (Testing) presents the testing strategy, unit test cases, integration test cases, and system-level test results in tabular format.

Chapter 8 (Results and Discussion) presents key system outputs and discusses results against project objectives.

Chapter 9 (Conclusion and Future Scope) summarizes contributions, discusses limitations, and outlines future enhancements.

References lists all sources in square-bracket format [1], [2], etc.

CHAPTER 2: LITERATURE SURVEY

2.1 Introduction

A literature survey is a systematic, critical examination of existing research, published works, and deployed systems relevant to the domain of the proposed project. Its purpose is to establish the current state of knowledge in the field, expose limitations of existing approaches, and position the proposed system as a meaningful advancement over the current state of the art.

The literature survey for the Smart Campus project is organized around three primary themes: (1) traditional student information systems and their documented shortcomings; (2) the application of AI, machine learning, and rule-based analytics to student performance prediction; and (3) the use of NLP and sentiment analysis in educational feedback processing. By examining these themes, the unique contributions of the Smart Campus system become clearly apparent.

2.2 Existing Systems and Related Work

2.2.1 Traditional Student Information Systems

Student Information Systems (SIS) have been deployed in educational institutions since the 1980s. Well-known examples include Banner by Ellucian, PeopleSoft Campus Solutions, and various government-mandated portals such as India's SAMARTH ERP. These platforms handle enrollment, grading, timetable management, and administrative record-keeping at scale, but they have been widely criticized in the literature for their poor usability, monolithic architecture, and complete absence of analytical or predictive capabilities [1].

Romero and Ventura (2017) surveyed SIS deployments across developing nations and found that the vast majority function purely as data repositories, with no built-in mechanism for monitoring student performance trends or generating early warnings for at-risk students [2]. Faculty and administrators in these environments must manually export data to spreadsheets and perform analysis independently — a process that is inconsistent, time-consuming, and often abandoned entirely when workloads are high.

The Smart Campus system is designed to move educational institutions from this reactive posture to a proactive one. By automatically computing and displaying failure risk labels, placement scores, and personalized study suggestions at every student login, the system ensures that both students and administrators have continuous, up-to-date awareness of academic standing — without requiring any manual data analysis.

2.2.2 AI-Based Student Performance Prediction

The application of machine learning to academic outcome prediction has grown significantly over the past decade. Hussain et al. (2019) demonstrated that decision tree classifiers trained on attendance, prior GPA, and demographic data could predict student failure with approximately 78% accuracy [3]. Baker and Inventado (2014) identified attendance and prior academic performance as the two most consistently predictive factors across multiple Educational Data Mining (EDM) studies [4] — findings that directly inform the input selection for the Smart Campus prediction model.

Aman and Anand (2020) applied deep neural networks to student dropout prediction using LMS log data and achieved higher accuracy, but at the cost of interpretability [5]. The authors noted that the neural network's decisions were opaque to non-technical stakeholders, making it difficult for faculty to explain to students why they had been flagged as at-risk. This interpretability problem with complex ML models is a critical concern in educational applications, where transparency and trust are essential.

The Smart Campus system deliberately chooses a transparent, rule-based approach over a black-box ML model. The prediction rules are expressed in plain English, directly visible in model.py, and easily understood by any stakeholder. Tanner and Allen (2016) showed that even simple rule-based early warning systems, when integrated into student-facing dashboards, produced measurable improvements in attendance and academic engagement [6] — validating the Smart Campus design philosophy that practical impact on student behavior matters more than marginal improvements in predictive accuracy.

2.2.3 Sentiment Analysis in Educational Feedback

Sentiment analysis — the automated classification of text into positive, negative, or neutral sentiment categories — has been applied in educational settings to analyze course evaluations, social media posts, and open-ended survey responses. Altrabsheh, Gaber, and Cocea (2014) developed SA-E, a real-time sentiment analysis system for student feedback,

using SVM and Naive Bayes classifiers to detect sentiment shifts during live lectures [7]. Their system demonstrated that automated sentiment classification could provide educators with near-real-time insight into the effectiveness of their teaching.

Rajput, Haider, and Ghani (2016) compared multiple lexicon-based sentiment tools — including TextBlob and VADER — on student feedback from online learning platforms [8]. They found TextBlob to be particularly effective for the short, informal feedback texts typical of web form submissions, and highlighted its zero training data requirement, Python integration, and ease of use as major advantages for projects where NLP is a supporting rather than primary feature. The Smart Campus system adopts TextBlob for exactly these reasons: its seamless integration with Python and Flask, its accuracy on short informal text, and the elimination of any need for training data collection and annotation.

Liu (2012) provided a comprehensive theoretical foundation for sentiment analysis, discussing the distinction between lexicon-based and machine-learning approaches, and establishing the three-class classification scheme (Positive, Neutral, Negative) as the standard for practical applications [9]. The Smart Campus feedback module directly implements this three-class scheme using TextBlob's polarity score, where polarity > 0 maps to Positive, polarity = 0 maps to Neutral, and polarity < 0 maps to Negative.

2.2.4 Role-Based Access Control in Academic Systems

Role-Based Access Control (RBAC) was formally described by Ferraiolo and Kuhn (1992) as a security paradigm in which access rights are granted to roles rather than individual users [10]. Sandhu et al. (1996) further developed RBAC into a family of reference models establishing role hierarchies and separation-of-duty constraints [11]. In educational systems, RBAC ensures that students access only their own data, faculty access student lists relevant to their responsibilities, and administrators have comprehensive system control.

The Smart Campus system implements a flat RBAC model with three roles stored as VARCHAR values in the users table. Upon login, the system reads the user's role and routes them to the appropriate dashboard. This role-based routing is performed in the login route handler in app.py, which branches based on the role value retrieved from the database. While simpler than the full RBAC3 model, this implementation provides all the access control granularity required for the system's current scope.

2.3 Drawbacks of Existing Systems

- Absence of Real-Time Analytics: Most existing SIS platforms provide historical data storage without any real-time computational analysis, prediction, or recommendation generation.
- No Personalized Recommendations: Existing systems provide no mechanism to generate individual study suggestions or academic guidance based on a student's specific performance profile.
- Wasted Feedback Data: Feedback collection is common, but systematic automated analysis of responses is rare, leaving valuable student insights unutilized.
- Data Silos: Student records are distributed across multiple disconnected systems, preventing holistic, cross-domain performance analysis.
- High Cost of Enterprise Solutions: Platforms like Banner and PeopleSoft are prohibitively expensive for smaller institutions and out of scope for student-developed projects.
- Interpretability Gaps in ML Systems: More sophisticated ML-based academic prediction systems often sacrifice transparency for marginal accuracy gains, undermining trust among educators and students.

2.4 Proposed System

The Smart Campus system is proposed as a lightweight, intelligent, and integrated web application that addresses all the identified limitations. It consolidates student marks, attendance, faculty profiles, and feedback into a single MySQL database and presents role-specific dashboards to all stakeholders. The prediction engine uses interpretable rule-based logic, and the feedback module uses TextBlob NLP for automatic sentiment classification — all implemented with free, open-source Python technologies.

Key advantages of the proposed system: (1) Unified data platform eliminating silos; (2) Real-time predictive analytics at every student login; (3) Transparent, interpretable prediction rules; (4) Automated feedback sentiment classification; (5) Role-based security and data privacy; (6) Zero software licensing cost; (7) Modular architecture enabling future ML model integration.

CHAPTER 3: SYSTEM ANALYSIS

3.1 Feasibility Study

A feasibility study is conducted before development to determine whether the proposed system is achievable within given constraints. The Smart Campus feasibility analysis covers three dimensions: technical, operational, and economic.

3.1.1 Technical Feasibility

All required technologies — Python 3.8+, Flask 2.x, MySQL 8.0, mysql-connector-python, and TextBlob — are freely available, open-source, and compatible with standard Windows, Linux, and macOS operating systems. No specialized hardware, proprietary software, or cloud services are required. Python is pre-installed on most Linux systems; all packages can be installed via pip with a single command. The development team has the required skills in Python, Flask, SQL, and basic NLP. The project is fully technically feasible.

3.1.2 Operational Feasibility

The system's web-based interface requires only a browser and basic computer literacy. Pages are designed to be intuitive, with clear labels, role-filtered content, and logical navigation. Students benefit from personalized insights; faculty benefit from convenient profile management; administrators gain automated feedback analysis. These advantages are expected to drive strong adoption. No installation or technical training is required on the client side. The project is operationally feasible.

3.1.3 Economic Feasibility

The Smart Campus system incurs zero software licensing costs, as all components are open-source. Deployment hardware requirements are minimal — a standard desktop PC or low-cost Linux server suffices. The primary cost is developer time. Benefits include reduced administrative workload, timely identification of at-risk students, and improved feedback utilization. Even a modest improvement in student pass rates would justify the development investment many times over. The project is economically feasible.

3.2 Functional Requirements

3.2.1 Authentication and Authorization

- FR-01: Provide a login form accepting username and password, accessible to all user types.
- FR-02: Validate credentials against the users table using parameterized SQL queries.
- FR-03: Determine user role from the database and redirect to the appropriate dashboard.
- FR-04: Return a descriptive error message for invalid credentials without revealing specifics.
- FR-05: Provide a logout route that ends the session and returns the user to the home page.

3.2.2 Student Module

- FR-06: Display student name, marks for DBMS/Python/AI, attendance percentage, and overall average.
- FR-07: Compute and display failure risk label: High Risk, Moderate Risk, or Low Risk.
- FR-08: Compute and display placement potential: High, Medium, or Low Placement Chance.
- FR-09: Generate and display a personalized study suggestion.
- FR-10: Provide a feedback text form for the student to submit open-ended feedback.

3.2.3 Feedback and Sentiment Analysis

- FR-11: Accept feedback text and student username; resolve student_id from the students table.
- FR-12: Classify feedback sentiment as Positive, Neutral, or Negative using TextBlob.
- FR-13: Store feedback text, sentiment, and student_id in the feedback table.
- FR-14: Display the sentiment classification result to the student as a submission confirmation.

3.2.4 Faculty Module

- FR-15: Display the complete student list when a faculty member logs in.
- FR-16: Show the faculty member's profile (name, department, email, phone, subject) on a profile page.
- FR-17: Allow faculty to update profile fields and persist changes to the faculty table.

3.2.5 Admin Module

- FR-18: Display the complete student list when an administrator logs in.
- FR-19: Allow the admin to select a student and view their marks, attendance, and overall percentage.
- FR-20: Allow the admin to update selected student marks and attendance; recompute and persist overall percentage.
- FR-21: Display all feedback entries and their sentiment labels for a selected student.

3.3 Non-Functional Requirements

3.3.1 Performance

- NFR-01: All page loads shall complete within 3 seconds under normal single-user conditions.
- NFR-02: Sentiment analysis of submitted feedback shall complete within 1 second.
- NFR-03: All database operations shall complete within 500 ms for expected dataset sizes.

3.3.2 Security

- NFR-04: All SQL queries shall use parameterized statements to prevent SQL injection.
- NFR-05: No cross-role data access shall be permitted; each user sees only role-appropriate data.
- NFR-06: Database credentials shall not be exposed in publicly accessible files.

3.3.3 Usability and Reliability

- NFR-07: Pages shall render correctly in Chrome 90+, Firefox 88+, and Edge 90+.
- NFR-08: All prediction outputs and sentiment results shall be expressed in plain, non-technical English.
- NFR-09: All database writes shall use db.commit() to ensure persistence after successful execution.
- NFR-10: The code shall follow a modular structure: app.py (routing), model.py (predictions), sentiment.py (NLP).

3.4 Hardware Requirements

Component	Minimum	Recommended
Processor	Intel Core i3, 1.6 GHz	Intel Core i5, 2.4 GHz+
RAM	4 GB	8 GB or more
Storage	20 GB free	50 GB SSD
Network	Standard LAN/Wi-Fi	100 Mbps LAN
Display	1024×768	1920×1080 Full HD
Input	Keyboard and mouse	Keyboard and mouse

Table 3.1: Hardware Requirements for Smart Campus System

3.5 Software Requirements

Component	Version / Details	Purpose
Operating System	Windows 10/11 or Ubuntu 20.04+	Host environment
Python	3.8 or higher	Primary language
Flask	2.x	Web framework and routing
mysql-connector-python	Latest (pip install)	MySQL database access
TextBlob	0.17.x	Sentiment analysis (NLP)
MySQL Server	8.0 or higher	Relational database engine
MySQL Workbench	8.0	Schema design and management
Web Browser	Chrome 90+, Firefox 88+	Client-side access
VS Code / PyCharm	Latest stable	Development environment
Git	2.x	Version control

Table 3.2: Software Requirements for Smart Campus System

CHAPTER 4: SYSTEM DESIGN

4.1 System Architecture

The Smart Campus system follows a three-tier client-server architecture that cleanly separates the application into Presentation, Application, and Data layers. This separation promotes maintainability, testability, and scalability — changes to one layer can be made independently of the others.

4.1.1 Presentation Layer

The Presentation Layer consists of five Jinja2 HTML templates: index.html (home/login page), student_dashboard.html, faculty_dashboard.html, faculty_profile.html, and admin_dashboard.html. Templates receive data from the Application Layer as Jinja2 context variables and render them as HTML pages in the user's browser. CSS styling provides clean visual formatting for all dashboards. Jinja2 syntax ({{ }}, {% %}) enables dynamic content rendering, conditional display of prediction results, and iteration over student and feedback lists.

4.1.2 Application Layer

The Application Layer comprises three Python files: app.py (Flask routes and main business logic), model.py (the rule-based prediction engine), and sentiment.py (the TextBlob NLP module). Flask receives HTTP requests from the browser, executes the appropriate handler, coordinates database queries through mysql-connector-python, calls model.py or sentiment.py functions as needed, and returns rendered HTML responses. This layer is stateless between requests in the current implementation.

4.1.3 Data Layer

The Data Layer is a MySQL 8.0 relational database named "smartcampus" with four tables: users, students, faculty, and feedback. All database access is performed exclusively through parameterized SQL queries in app.py, ensuring both security and data integrity. The database is accessed over a local TCP connection via mysql-connector-python.

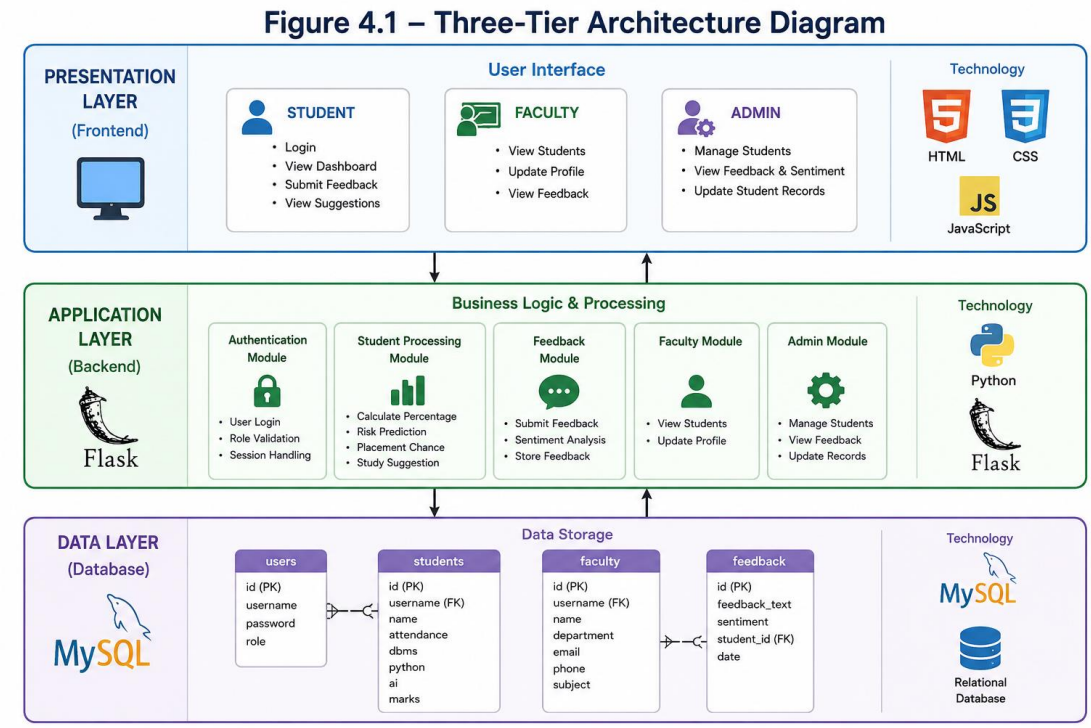


Fig 4.1: Three-Tier Architecture of the Smart Campus System

4.2 Data Flow Diagrams

4.2.1 Level 0 DFD — Context Diagram

The Context Diagram represents the entire Smart Campus system as a single process and shows the three external entities (Student, Faculty, Admin) and the high-level data flows between them and the system. Student inputs: credentials, feedback text. Student outputs: academic dashboard (marks, attendance, percentage, risk, placement, suggestion), sentiment result. Faculty inputs: credentials, profile update data. Faculty outputs: student list, own profile data. Admin inputs: credentials, student selection, record update data. Admin outputs: student records, sentiment-labeled feedback.

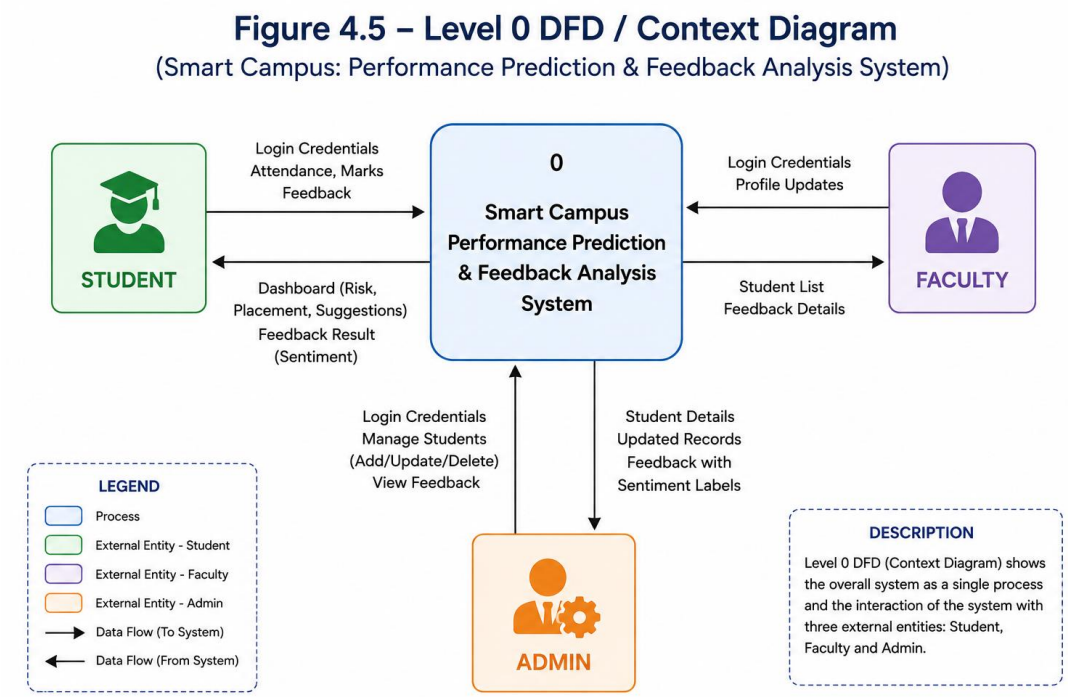


Fig 4.2: Context Diagram showing external entities and top-level data flows

4.2.2 Level 1 DFD

The Level 1 DFD decomposes the system into five sub-processes with four data stores: D1 (users table), D2 (students table), D3 (faculty table), D4 (feedback table). Process 1.0 — Authentication: reads D1, validates credentials, routes to appropriate process. Process 2.0 — Student Dashboard: reads D2, computes percentage, calls model functions, returns dashboard. Process 3.0 — Feedback Processing: reads D2 for student_id, invokes TextBlob, writes to D4, returns sentiment. Process 4.0 — Faculty Operations: reads D2 (student list); reads/writes D3 (profile). Process 5.0 — Admin Operations: reads D2, D4; writes D2 (record updates).

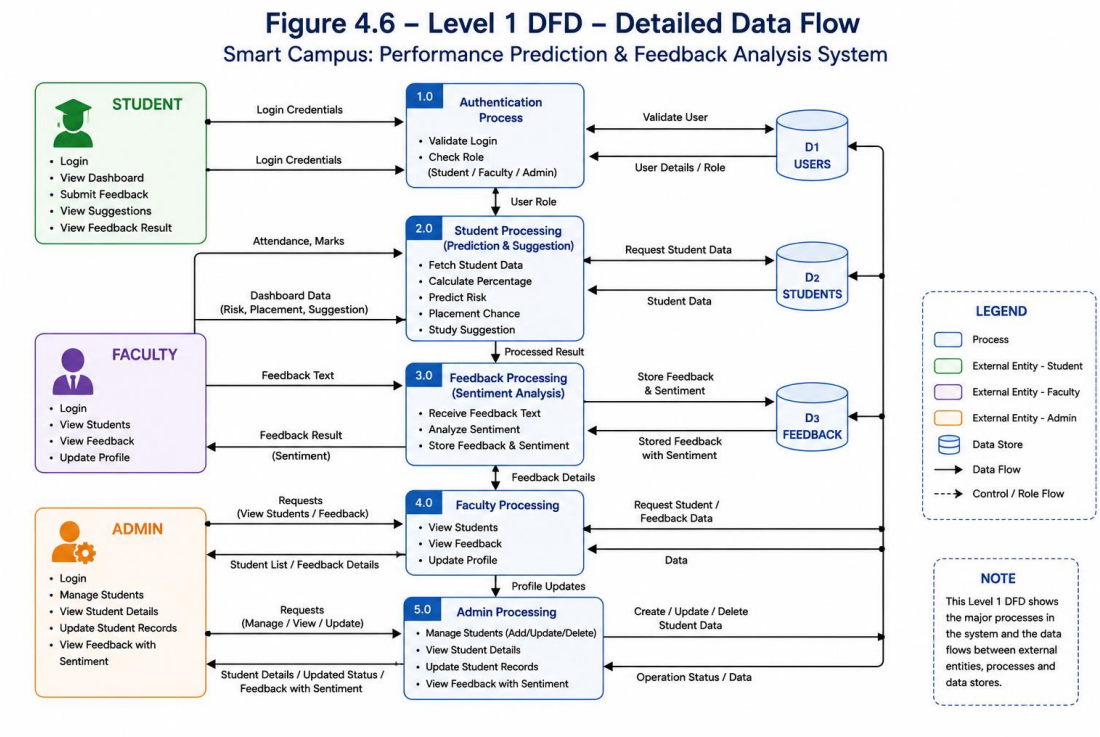


Fig 4.3: Level 1 DFD showing all sub-processes (P1.0–P5.0) and data stores (D1–D4)

4.3 Use Case Diagram

The Use Case Diagram shows the interactions between the three actors and the system's use cases. Student use cases: Login, View Dashboard, View Failure Risk, View Placement Score, View Study Suggestion, Submit Feedback, View Sentiment Result, Logout. Faculty use cases: Login, View Student List, View Profile, Update Profile, Logout. Admin use cases: Login, View Student List, Select Student Details, Update Student Records, View Sentiment-Labeled Feedback, Logout. Shared: Login and Logout (all actors); View Student List (Faculty + Admin).

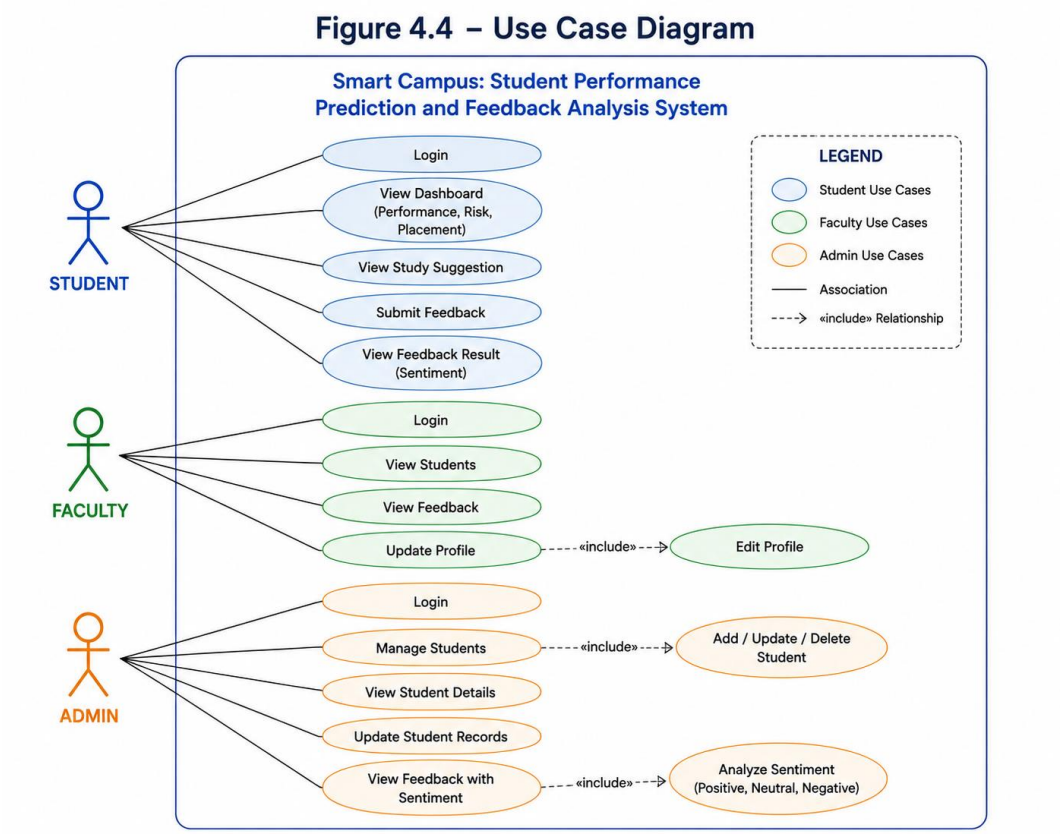


Fig 4.4: Use Case Diagram for the Smart Campus System

4.4 Database Design

4.4.1 Entity-Relationship (ER) Diagram

Entities: User (PK: username; attributes: password, role), Student (PK: id; attributes: name, username[FK], attendance, dbms, python, ai, marks), Faculty (PK: username[FK]; attributes: name, department, email, phone, subject), Feedback (PK: id; attributes: feedback_text, sentiment, student_id[FK]). Relationships: User ONE-TO-ONE Student (via username FK); User ONE-TO-ONE Faculty (via username FK); Student ONE-TO-MANY Feedback (via student_id FK).

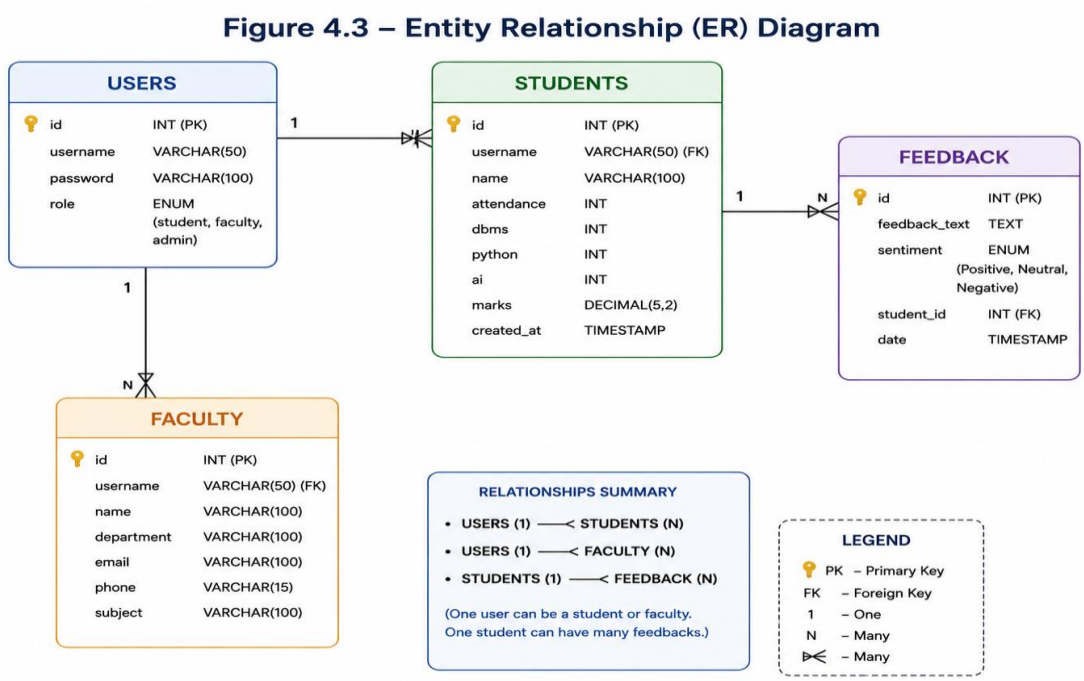


Fig 4.5: ER Diagram showing all entities, attributes, and relationships

4.4.2 Table: users

Column	Data Type	Constraints	Description
username	VARCHAR(50)	PRIMARY KEY	Unique login identifier
password	VARCHAR(100)	NOT NULL	User login password
role	VARCHAR(20)	NOT NULL	student / faculty / admin

Table 4.1: users table structure

4.4.3 Table: students

Column	Data Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Unique student ID
name	VARCHAR(100)	NOT NULL	Student full name
username	VARCHAR(50)	FK → users	Login reference
attendance	INT	DEFAULT 0	Attendance percentage
dbms	INT	DEFAULT 0	DBMS marks (0–100)
python	INT	DEFAULT 0	Python marks (0–100)
ai	INT	DEFAULT 0	AI marks (0–100)
marks	FLOAT	DEFAULT 0	Computed overall average

Table 4.2: students table structure

4.4.4 Table: faculty

Column	Data Type	Constraints	Description
username	VARCHAR(50)	PK, FK → users	Login reference
name	VARCHAR(100)	NOT NULL	Full name
department	VARCHAR(100)	NOT NULL	Department name
email	VARCHAR(100)	NOT NULL	Email address
phone	VARCHAR(15)	NOT NULL	Phone number
subject	VARCHAR(100)	NOT NULL	Subject taught

Table 4.3: faculty table structure

4.4.5 Table: feedback

Column	Data Type	Constraints	Description
id	INT	PK, AUTO_INCREMENT	Unique feedback ID
feedback_text	TEXT	NOT NULL	Student feedback content
sentiment	VARCHAR(20)	NOT NULL	Positive / Neutral / Negative
student_id	INT	FK → students(id)	Submitting student reference

Table 4.4: feedback table structure

4.5 Module Design

The Smart Campus system is organized into six functional modules. Each module maps to one or more Flask routes and has clearly defined responsibilities:

4.5.1 Authentication Module

Routes: GET / (login page), POST /login (validation), GET /logout. Responsibilities: Accept and validate credentials, determine user role, route to appropriate dashboard, handle invalid login gracefully, and reset session on logout.

4.5.2 Student Dashboard Module

Route: POST /login (student branch). Responsibilities: Retrieve student record, compute overall percentage as round((dbms+python+ai)/3, 2), invoke model.predict_failure(), model.placement_score(), and model.study_suggestion(), pass all values to student_dashboard.html.

4.5.3 Feedback and Sentiment Analysis Module

Route: POST /feedback. Responsibilities: Retrieve feedback text and student username, resolve student_id, call sentiment.analyze_feedback(text), INSERT result into feedback table, commit, return sentiment confirmation.

4.5.4 Faculty Module

Routes: POST /login (faculty branch), GET /faculty_profile/<username>, POST /update_faculty. Responsibilities: Retrieve and render student list for faculty dashboard; retrieve and render faculty profile; accept and validate profile update; execute UPDATE on faculty table.

4.5.5 Admin Module

Routes: POST /login (admin branch), POST /admin_student, POST /update_student. Responsibilities: Render student list; retrieve selected student data + feedback; accept and validate record updates; compute new overall percentage; execute UPDATE on students table.

4.5.6 Prediction Model Module (model.py)

Functions: predict_failure(attendance, marks), placement_score(marks, attendance), study_suggestion(marks, attendance). All are pure functions with no side effects. Called from app.py student dashboard handler. Rule thresholds: attendance < 60 OR marks < 40 → High Risk; marks < 60 → Moderate Risk; otherwise → Low Risk. Score = (marks + attendance)/2: >80 → High Placement, >60 → Medium, else → Low. Study suggestion based on marks < 50 and/or attendance < 70 condition combinations.

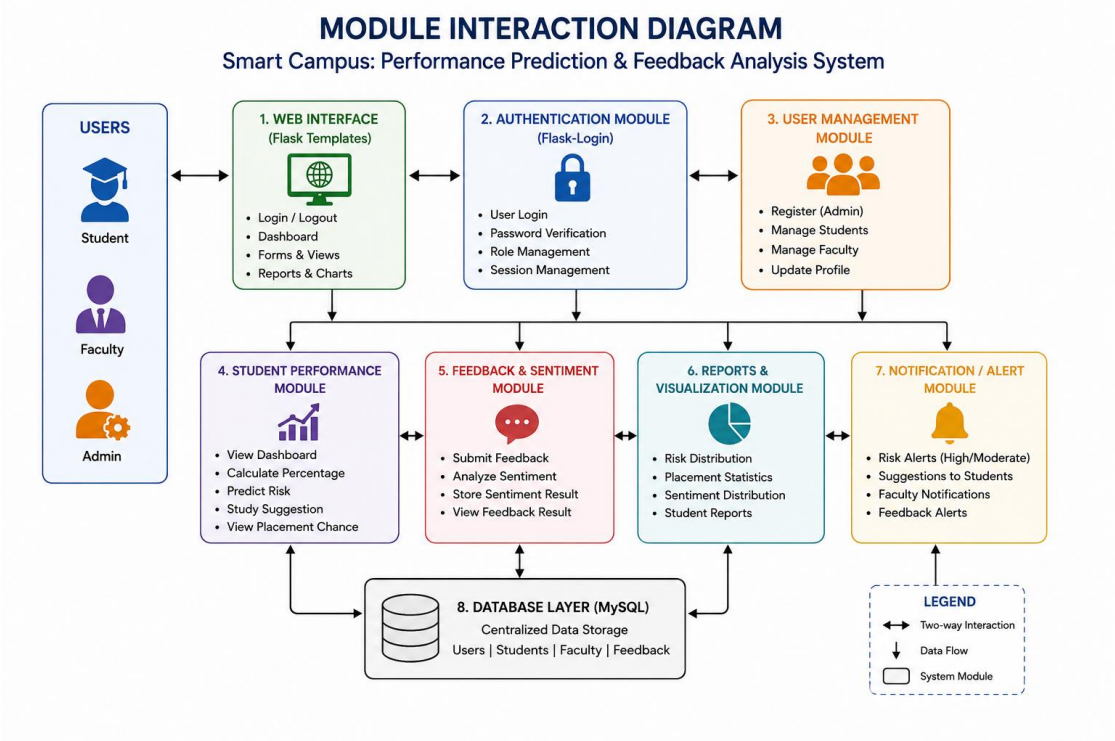


Fig 4.6: Module Interaction Diagram showing data flow between all system modules

4.6 Sequence Diagrams

4.6.1 Student Login and Dashboard Rendering

Step 1: Browser sends POST /login (username, password) to Flask app.py. Step 2: app.py executes SELECT role FROM users WHERE username=%s AND password=%s on D1 (users table). Step 3: MySQL returns role = "student". Step 4: app.py executes SELECT name, attendance, dbms, python, ai FROM students WHERE username=%s on D2. Step 5: app.py computes percentage = round((dbms+python+ai)/3, 2). Step 6: app.py calls predict_failure(attendance, percentage) → risk string. Step 7: app.py calls placement_score(percentage, attendance) → placement string. Step 8: app.py calls study_suggestion(percentage, attendance) → suggestion string. Step 9: app.py calls render_template("student_dashboard.html", ...). Step 10: Jinja2 renders HTML. Step 11: Flask returns rendered page to browser.

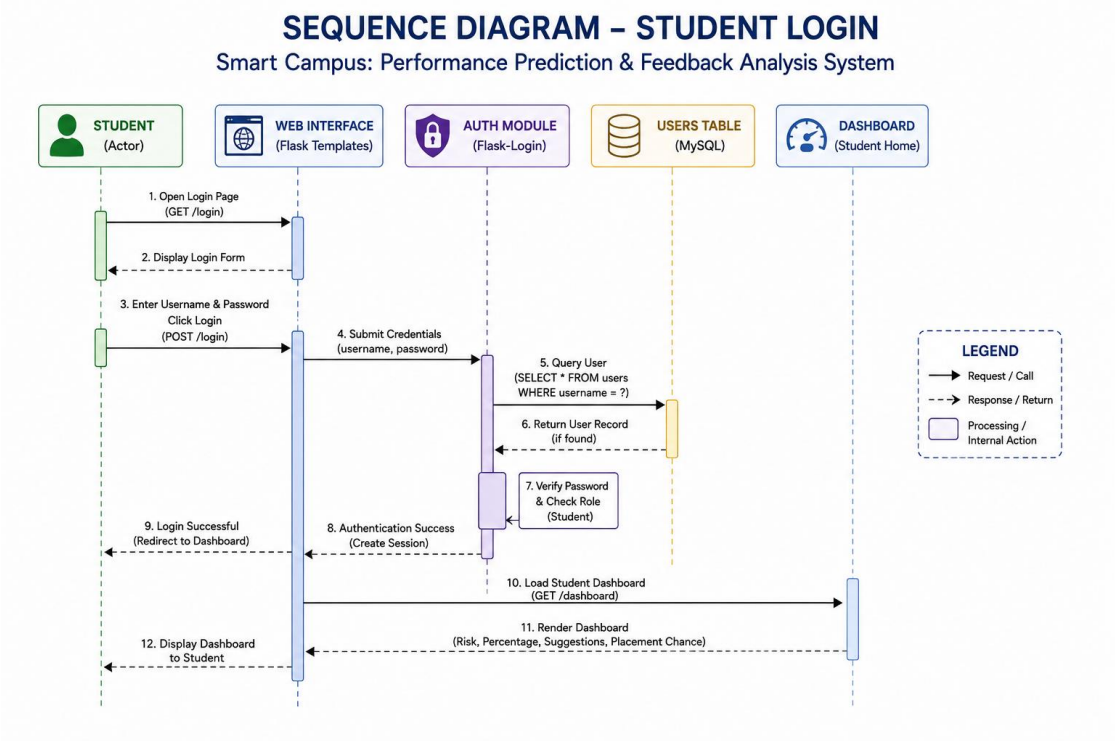


Fig 4.7: Sequence Diagram showing complete student login interaction

4.6.2 Feedback Submission and Sentiment Analysis

Step 1: Student submits feedback form; browser sends POST /feedback (feedback, username) to app.py. Step 2: app.py executes SELECT id FROM students WHERE username=%s to resolve student_id. Step 3: app.py calls sentiment.analyze_feedback(text). Step 4: sentiment.py creates TextBlob(text), evaluates polarity, returns "Positive", "Neutral", or "Negative". Step 5: app.py executes INSERT INTO feedback (feedback_text, sentiment, student_id) VALUES (%s, %s, %s). Step 6: db.commit() persists the record. Step 7: app.py returns "Feedback Sentiment: " + result to browser. Step 8: Browser displays sentiment confirmation to student.

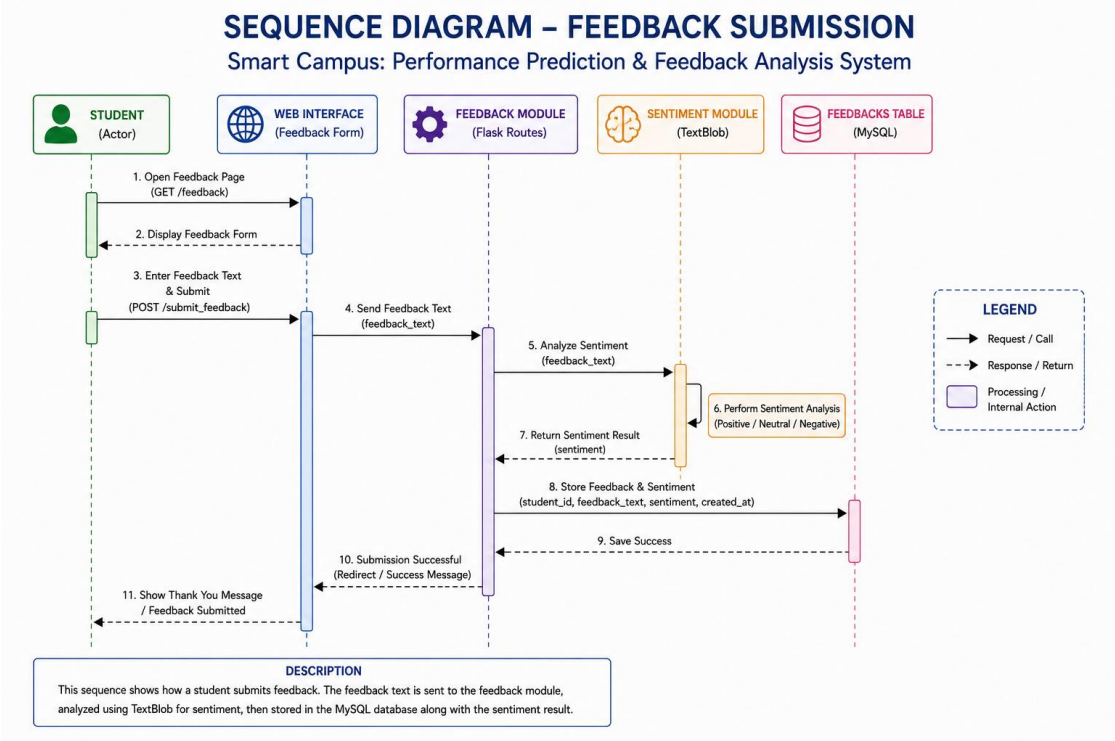


Fig 4.8: Sequence Diagram showing complete feedback submission interaction

CHAPTER 5: TECHNOLOGY STACK

5.1 Python Programming Language

Python is a high-level, interpreted, dynamically typed, general-purpose programming language renowned for its clean syntax, readability, and expansive ecosystem of third-party libraries. Created by Guido van Rossum and first released in 1991, Python has grown to become one of the most widely used programming languages globally, consistently ranking at the top of the TIOBE Community Index, the IEEE Spectrum language rankings, and the Stack Overflow Developer Survey. Its design philosophy — often summarized as "The Zen of Python" — emphasizes simplicity, clarity, and the principle that there should be one obvious, elegant way to accomplish each programming task.

Python's versatility is a key reason for its selection as the primary language for the Smart Campus project. The language supports web development (Flask, Django), data science (NumPy, Pandas, scikit-learn), natural language processing (TextBlob, NLTK, spaCy), and database interaction (mysql-connector-python, SQLAlchemy) — all domains that the Smart Campus project touches simultaneously. Python 3.8+ is used in this project, providing all modern language features required by the project's dependencies, including improved f-string support, type hints, the walrus operator, and enhanced async capabilities.

The Python Package Index (PyPI) hosts over 450,000 open-source packages, all installable via pip. For Smart Campus, the complete development environment can be set up with three pip install commands: pip install flask, pip install mysql-connector-python, and pip install textblob — followed by python -m textblob.download_corpora to download the NLTK language data that TextBlob requires. This minimal setup overhead makes Python an excellent choice for projects that must be developed and deployed quickly on diverse hardware configurations.

5.2 Flask Web Framework

Flask is a lightweight, micro web framework for Python, created by Armin Ronacher and released in 2010. The term "micro" reflects Flask's minimalist philosophy: the framework provides the essential primitives for building web applications — URL routing, HTTP request handling, response generation, and template rendering — without imposing decisions about which libraries to use for database access, user authentication, or other

concerns. This deliberate minimalism gives developers full control over the application stack and makes Flask exceptionally well-suited for projects that require fine-grained customization, such as the Smart Campus system.

Flask is built on two key dependencies: Werkzeug (a mature WSGI utility library handling HTTP protocol details, URL routing, and the built-in development server) and Jinja2 (a full-featured server-side templating engine). Both are independently maintained, production-tested libraries used by thousands of web applications worldwide. Flask itself adds the @app.route() decorator-based routing system and the render_template() / request / app application context that Python developers interact with directly.

5.2.1 Routing with @app.route()

Flask's decorator-based routing is one of its most distinctive and readable features. Each URL pattern is mapped to a Python handler function by placing the @app.route(path, methods=[...]) decorator immediately above it. The Smart Campus application defines eight routes: "/" (GET, renders login page), "/login" (POST, validates credentials and routes to dashboard), "/faculty_profile/<username>" (GET, dynamic route using URL parameter), "/admin_student" (POST, retrieves selected student data), "/update_student" (POST, updates student academic record), "/feedback" (POST, submits and analyzes feedback), "/update_faculty" (POST, updates faculty profile), and "/logout" (GET, ends session).

The dynamic URL parameter "<username>" in "/faculty_profile/<username>" is a particularly useful Flask feature. When Flask matches a request URL against this pattern, it extracts the value between the slashes and passes it as the keyword argument "username" to the handler function. This allows the system to construct a unique, directly-linkable URL for each faculty member's profile page, making profile links sharable and bookmarkable.

5.2.2 Request Handling and Jinja2 Templates

Flask's request object provides programmatic access to all data associated with the current HTTP request. In Smart Campus, request.form is used extensively to retrieve form-submitted data: request.form["username"] and request.form["password"] for login authentication, request.form["feedback"] and request.form["username"] for feedback submission, and request.form["student_id"], request.form["dbms"], request.form["python"], request.form["ai"], and request.form["attendance"] for the admin record update operation.

The render_template() function combines a Jinja2 HTML template file with a Python dictionary of data values (passed as keyword arguments) and returns the rendered HTML as an HTTP response. Data such as student name, marks, attendance, risk label, placement score, and study suggestion is passed from app.py to student_dashboard.html via this function, and rendered using Jinja2's {{ variable }} syntax. Conditional sections ({% if risk == "High Risk" %}...{% endif %}) and loops ({% for student in students %}...{% endfor %}) enable fully dynamic, data-driven page rendering.

5.3 MySQL Relational Database and mysql-connector-python

MySQL is an open-source relational database management system (RDBMS) that stores data in structured tables with predefined schemas. First released in 1995 and now maintained by Oracle Corporation under the MySQL Community Edition (GPL) license, MySQL is one of the most widely deployed databases in the world, used by applications ranging from personal projects to global enterprise systems. Its proven reliability, strong SQL compliance, comprehensive indexing and query optimization capabilities, and extensive documentation make it an excellent choice for the Smart Campus database backend.

The relational data model is particularly appropriate for Smart Campus, where data is inherently structured and relational: each student record is linked to a user record through a username foreign key, and each feedback entry is linked to a student record through a student_id foreign key. MySQL enforces these referential integrity constraints through FOREIGN KEY declarations in the CREATE TABLE statements, ensuring that no orphaned records can exist in the database — a critical data quality guarantee in a system where students, users, and feedback must always be consistent.

The mysql-connector-python library — the official Oracle-maintained Python driver for MySQL — provides a PEP 249-compliant Python interface for all database operations. The connection is established with mysql.connector.connect(host, user, password, database), and a buffered cursor (db.cursor(buffered=True)) is used to ensure all result rows are fetched immediately after query execution. Critically, all SQL queries use the parameterized %s placeholder syntax with a tuple of values, preventing SQL injection. For example: cursor.execute("SELECT role FROM users WHERE username=%s AND password=%s", (username, password)). The db.commit() method is called after every INSERT and UPDATE operation to persist changes.

5.4 TextBlob NLP Library

TextBlob is an open-source Python library for Natural Language Processing (NLP), providing a clean, intuitive API for common text processing tasks including tokenization, part-of-speech tagging, noun phrase extraction, translation, spell correction, and — most importantly for Smart Campus — sentiment analysis. Built on top of NLTK and the Pattern library, TextBlob combines their linguistic resources into an accessible, two-line API pattern that requires no machine learning expertise to use effectively.

TextBlob's sentiment analysis is based on the Pattern library's sentiment lexicon — a dictionary of English words each annotated with a polarity score (from -1.0 most negative, to +1.0 most positive) and a subjectivity score. When TextBlob.sentiment.polarity is computed for a piece of text, TextBlob tokenizes the text, looks up each recognized word in the lexicon, and computes the weighted average polarity while accounting for intensifiers (words like "very" that amplify sentiment), negations (words like "not" that invert it), and multi-word idioms. The resulting scalar polarity value is then used in Smart Campus's classify_feedback() function to label the feedback as Positive (>0), Neutral (=0), or Negative (<0).

5.4.1 Implementation in sentiment.py

The sentiment module (sentiment.py) contains a single function, analyze_feedback(text), which accepts a string of feedback text and returns one of three labels. The function instantiates a TextBlob object with the input text: analysis = TextBlob(text). It then accesses analysis.sentiment.polarity, a float in [-1.0, 1.0]. If polarity > 0, the function returns "Positive". If polarity == 0, it returns "Neutral". If polarity < 0, it returns "Negative". This three-class classification is stored in the database alongside the original feedback text, giving administrators an immediately interpretable summary of student sentiment for every submission.

TextBlob is installed via: pip install textblob, followed by python -m textblob.download_corpora to download the required NLTK language data files. This two-step installation is all that is required to enable full NLP-based sentiment analysis in the Smart Campus system — a testament to TextBlob's ease of use and minimal setup overhead.

5.4.2 Advantages of TextBlob for Smart Campus

- Zero Training Data Requirement: TextBlob uses a pre-built sentiment lexicon. No data collection, annotation, or model training is required.
- Instantaneous Results: Lexicon-based analysis produces results in milliseconds with no model inference latency.
- Good Accuracy on Short Informal Text: TextBlob performs particularly well on the short, informal sentences typical of web-form student feedback.
- Simple Python Integration: The complete implementation requires two import statements and three lines of code.
- Free and Open Source: No licensing costs, actively maintained on GitHub with extensive community documentation.

5.5 HTML, CSS, and Jinja2 Frontend

The Smart Campus frontend is built with standard web technologies: HTML5 for page structure, CSS for visual styling, and Jinja2 for server-side dynamic rendering. All five templates (index.html, student_dashboard.html, faculty_dashboard.html, faculty_profile.html, admin_dashboard.html) are stored in the templates/ directory and loaded by Flask's render_template() function.

HTML5 semantic elements — forms, tables, headings, paragraphs, input fields, and buttons — define the logical structure of each page. CSS provides visual formatting: clean typography, clearly bordered data tables, color-coded risk labels (red for High Risk, orange for Moderate Risk, green for Low Risk), and well-spaced form layouts. Jinja2 syntax enables dynamic content: {{ student_name }} renders the student's name, {{ risk }} renders the computed risk label, {% if %} blocks conditionally display alert messages, and {% for student in students %} loops render each row of the student list table.

The frontend deliberately avoids complex JavaScript frameworks to maintain simplicity and ensure that the system functions correctly in any modern browser without any client-side dependencies. All interactivity is driven by standard HTML form submissions with HTTP POST requests, handled server-side by Flask route handlers.

5.6 Development Tools

- Visual Studio Code: Primary code editor with Python extension for syntax highlighting, IntelliSense, and integrated terminal.

- MySQL Workbench 8.0: Graphical interface for database design, SQL scripting, and table inspection.
- pip: Python package manager for installing all dependencies.
- Git 2.x: Version control for tracking code changes throughout development.
- Flask Development Server (app.run(debug=True)): Built-in server with auto-reload and browser error tracebacks for development testing.

CHAPTER 6: IMPLEMENTATION

6.1 Overview of Implementation

This chapter describes the detailed implementation of every module of the Smart Campus system. Development followed an incremental approach: the MySQL database was created first, followed by the authentication module, student dashboard and prediction engine, faculty module, admin module, and finally the feedback and sentiment analysis module. Each component was individually tested before integration. The final source code is organized as: app.py (Flask application, 258 lines), model.py (prediction engine, 37 lines), sentiment.py (NLP module, 13 lines), and a templates/ folder with five HTML files.

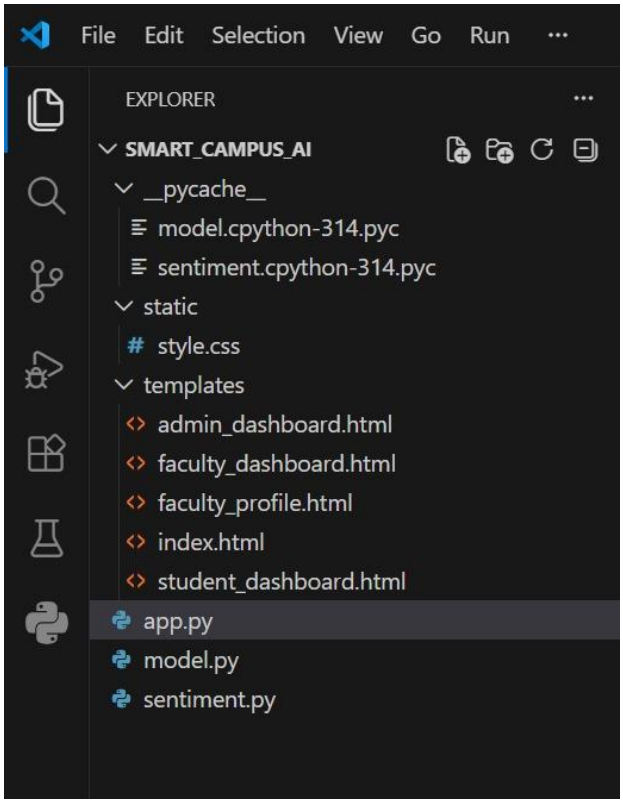


Fig 6.1: Smart Campus project folder structure as displayed in Visual Studio Code

6.2 Database Implementation

The smartcampus MySQL database was designed and created using MySQL Workbench. The following Data Definition Language (DDL) statements establish the complete database schema with all four tables and their relational constraints:

```
CREATE DATABASE smartcampus;
```

```
USE smartcampus;

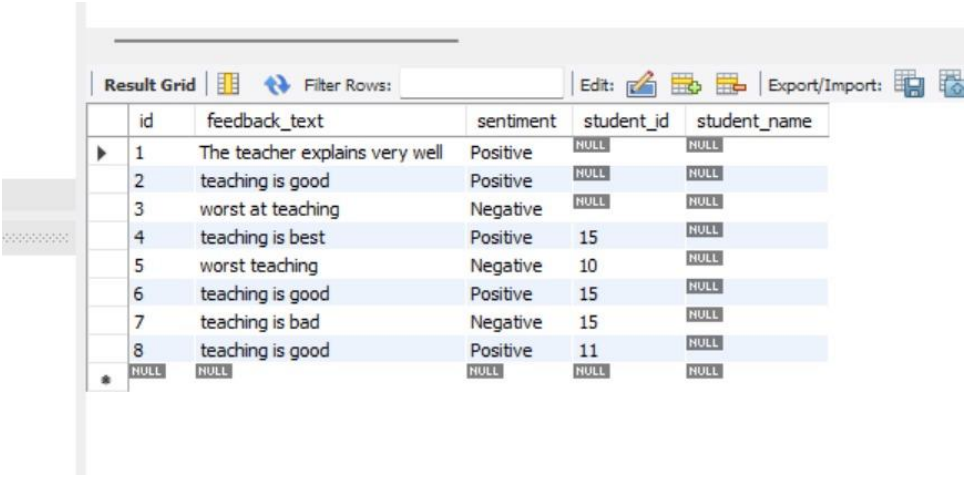
CREATE TABLE users (
    username  VARCHAR(50)  PRIMARY KEY,
    password  VARCHAR(100) NOT NULL,
    role      VARCHAR(20)  NOT NULL
);

CREATE TABLE students (
    id         INT          PRIMARY KEY AUTO_INCREMENT,
    name       VARCHAR(100) NOT NULL,
    username   VARCHAR(50),
    attendance INT          DEFAULT 0,
    dbms       INT          DEFAULT 0,
    python     INT          DEFAULT 0,
    ai         INT          DEFAULT 0,
    marks      FLOAT        DEFAULT 0,
    FOREIGN KEY (username) REFERENCES users(username)
);

CREATE TABLE faculty (
    username   VARCHAR(50)  PRIMARY KEY,
    name       VARCHAR(100) NOT NULL,
    department VARCHAR(100),
    email      VARCHAR(100),
    phone      VARCHAR(15),
    subject    VARCHAR(100),
    FOREIGN KEY (username) REFERENCES users(username)
);

CREATE TABLE feedback (
    id            INT  PRIMARY KEY AUTO_INCREMENT,
    feedback_text TEXT NOT NULL,
    sentiment     VARCHAR(20) NOT NULL,
    student_id    INT,
    FOREIGN KEY (student_id) REFERENCES students(id)
);
```

After creating the schema, representative test data was inserted via MySQL Workbench for all three user roles — student, faculty, and admin — along with corresponding entries in the students and faculty tables. This test dataset enabled systematic testing of every route and every prediction scenario.



id	feedback_text	sentiment	student_id	student_name
1	The teacher explains very well	Positive	NULL	NULL
2	teaching is good	Positive	NULL	NULL
3	worst at teaching	Negative	NULL	NULL
4	teaching is best	Positive	15	NULL
5	worst teaching	Negative	10	NULL
6	teaching is good	Positive	15	NULL
7	teaching is bad	Negative	15	NULL
8	teaching is good	Positive	11	NULL
NULL	NULL	NULL	NULL	NULL

Fig 6.2: MySQL Workbench view of the smartcampus database with all four tables

6.3 Application Initialization (app.py)

The file app.py begins by importing Flask and its utilities, along with mysql.connector, and the two local modules model and sentiment. The Flask application is instantiated with app = Flask(__name__). The MySQL connection is established at startup using mysql.connector.connect() with the host ("localhost"), user ("root"), password, and database ("smartcampus") parameters. A buffered cursor is created immediately — buffered=True ensures results are fully fetched before any subsequent query is run on the same connection, preventing "Unread result found" errors that would occur with unbuffered cursors on multi-query workflows.

The use of a single persistent connection and cursor (rather than a per-request connection pool) is appropriate for a single-developer testing environment. In a production deployment serving multiple concurrent users, a connection pool — implemented via SQLAlchemy's connection pooling or Flask-SQLAlchemy — would be used to prevent resource contention and ensure thread safety.

6.4 Home Page and Login Interface

The home page route (GET "/") is the simplest route in the application, serving a single purpose: rendering the index.html template, which presents the login form to the user. The login form contains two input fields (username and password) and a submit button, and sends an HTTP POST request to the "/login" route upon submission.

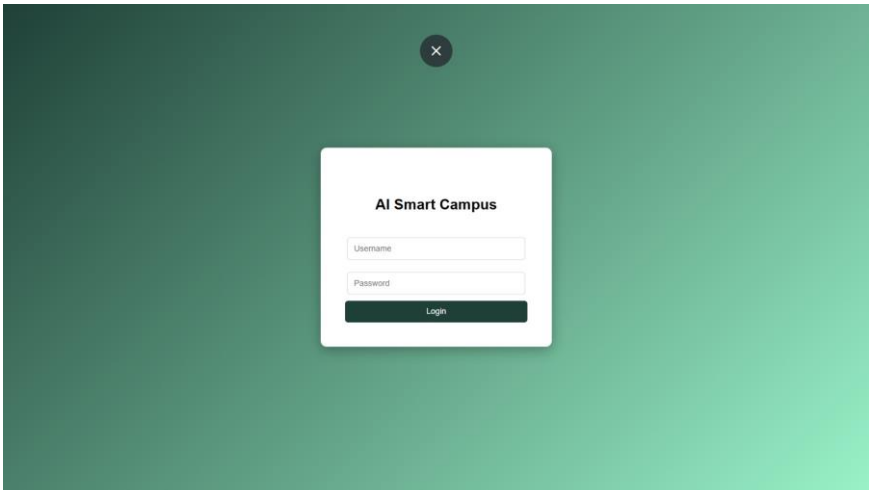


Fig 6.3: Smart Campus home page displaying the login form (index.html)

6.5 Authentication Module

The POST /login route handler is the most central function in app.py, as it serves all three user roles. The handler retrieves username and password from request.form and executes a parameterized SELECT query on the users table. If no matching record is found, the handler returns the string "Invalid Login". If credentials match, the handler reads the role value from the returned row and branches into one of three code paths.

For the Student role: the handler fetches the student's marks and attendance from the students table, computes the overall percentage, calls all three model.py prediction functions, and renders student_dashboard.html with all computed values. For the Faculty role: the handler fetches the complete student list and renders faculty_dashboard.html. For the Admin role: the handler fetches the student list and renders admin_dashboard.html with an initially empty student_data and feedbacks list (waiting for the admin to select a student).

The branching is implemented with a simple if-elif-else structure checking the role string value. This approach is clear, easy to understand, and straightforward to extend — adding a new role requires only the addition of a new elif branch with the appropriate data retrieval and template rendering logic.

6.6 Student Dashboard Module

Upon successful student login, the dashboard handler retrieves five fields from the students table: name, attendance, dbms, python, ai. The overall percentage is computed as: percentage = round((dbms + python_marks + ai) / 3, 2). Integer conversion with or-0

default (int(data[1] or 0)) handles NULL database values gracefully, preventing runtime exceptions when a field has not been populated.

The three computed prediction values — risk, placement, and suggestion — are generated by calling model.predict_failure(attendance, percentage), model.placement_score(percentage, attendance), and model.study_suggestion(percentage, attendance) respectively. All seven values (name, username, attendance, dbms, python, ai, percentage, risk, placement, suggestion) are then passed to render_template("student_dashboard.html", ...) as keyword arguments, making them available to the Jinja2 template for rendering.

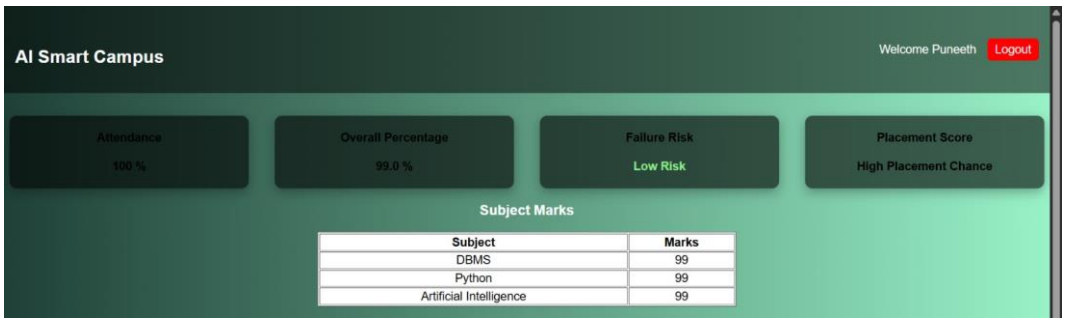


Fig 6.4: Student dashboard displaying subject-wise marks and attendance percentage

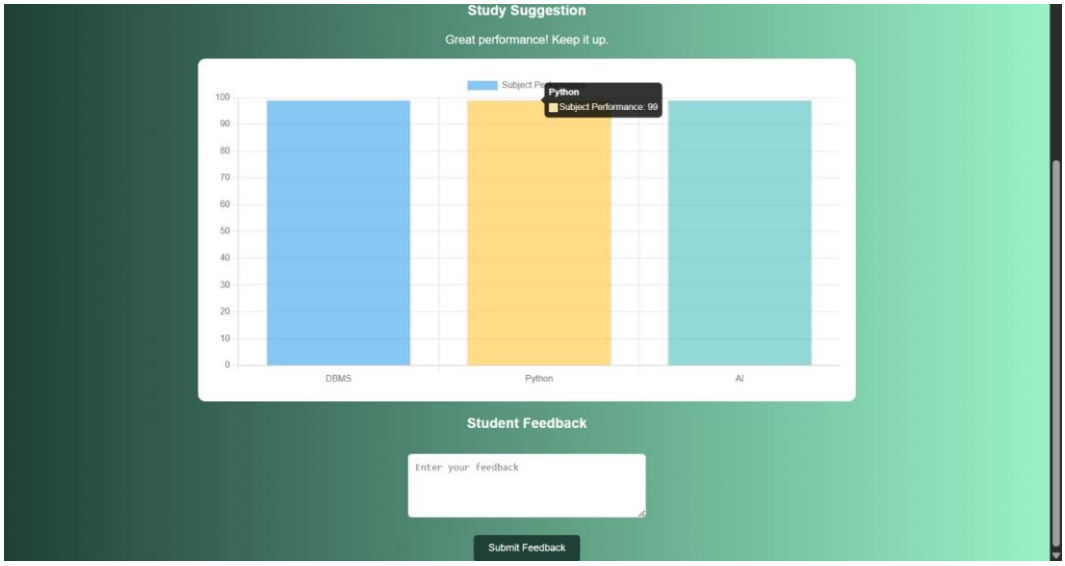


Fig 6.5: Student dashboard showing AI-generated failure risk prediction, placement score, and study suggestion

6.7 Prediction Model Module (model.py)

The prediction engine in model.py contains three pure functions, each implementing a distinct rule-based decision process. The functions are "pure" in the sense that they have

no side effects: they do not read from or write to any external resource (database, file, network), and their output is determined entirely by their input parameters. This purity makes them trivially testable in isolation and easy to replace with ML-based implementations in future versions.

6.7.1 predict_failure(attendance, marks)

This function classifies a student's academic risk level based on two numeric inputs: attendance (an integer percentage) and marks (a float average percentage). The decision logic applies the following ordered conditions: If attendance < 60 OR marks < 40 → return "High Risk" (the student has critically low attendance or critically low academic performance, or both; immediate intervention is required). Else if marks < 60 → return "Moderate Risk" (attendance is acceptable but academic performance needs improvement). Else → return "Low Risk" (both attendance and marks meet satisfactory thresholds; the student is performing well).

The OR condition in the High Risk branch is significant: a student with adequate marks (e.g., 65%) but attendance below 60% is still classified as High Risk, reflecting the reality that poor attendance typically predicts declining performance in subsequent assessments and makes the student ineligible for examinations in many college regulations.

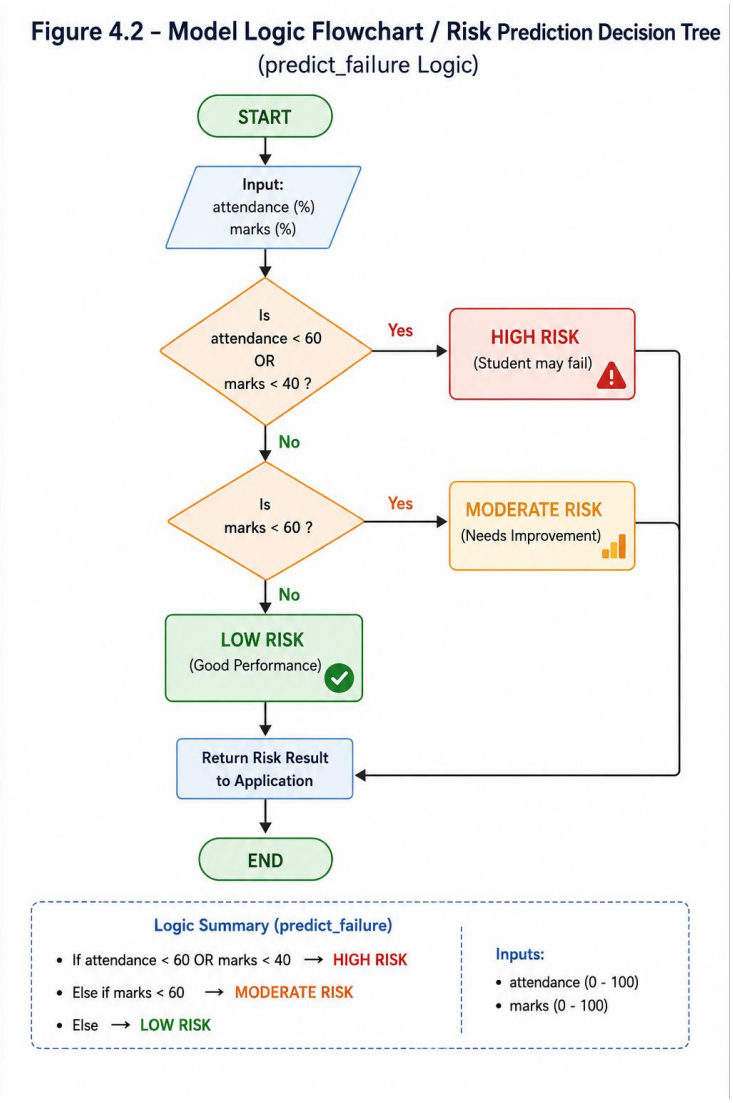


Fig 6.6: Flowchart showing the decision logic of the predict_failure() function

6.7.2 placement_score(marks, attendance)

This function estimates a student's campus placement readiness by computing a composite score as the simple average of marks and attendance: score = (marks + attendance) / 2. The composite score is then classified: score > 80 → "High Placement Chance" (the student has strong combined academic performance and attendance, making them an attractive candidate for campus recruiters); score > 60 → "Medium Placement Chance" (the student is a viable candidate but has room for improvement); else → "Low Placement Chance" (significant improvement in both marks and attendance is needed).

The composite score approach reflects the reality that campus recruiters evaluate candidates on both academic performance (marks) and discipline indicators (attendance). A student who excels academically but has poor attendance may still be at a disadvantage during

campus recruitment, as many companies use attendance as a proxy for workplace punctuality and discipline.

6.7.3 study_suggestion(marks, attendance)

This function generates a plain-language, actionable recommendation for the student based on which of the two performance thresholds (marks < 50, attendance < 70) are not met. Four mutually exclusive cases are handled: (1) marks < 50 AND attendance < 70 → "Increase study hours and attend more classes." (Both are below threshold; improvement is needed on both fronts.) (2) marks < 50 (attendance is adequate) → "Focus more on studying and revise difficult topics." (Academic effort is the primary concern.) (3) attendance < 70 (marks are adequate) → "Improve attendance to understand subjects better." (Class participation needs attention.) (4) Both adequate → "Great performance! Keep it up." (Positive reinforcement for good performers.)

Figure 5.3 – Flowchart: Study Suggestion Decision Logic

(Based on study_suggestion(marks, attendance) Logic)

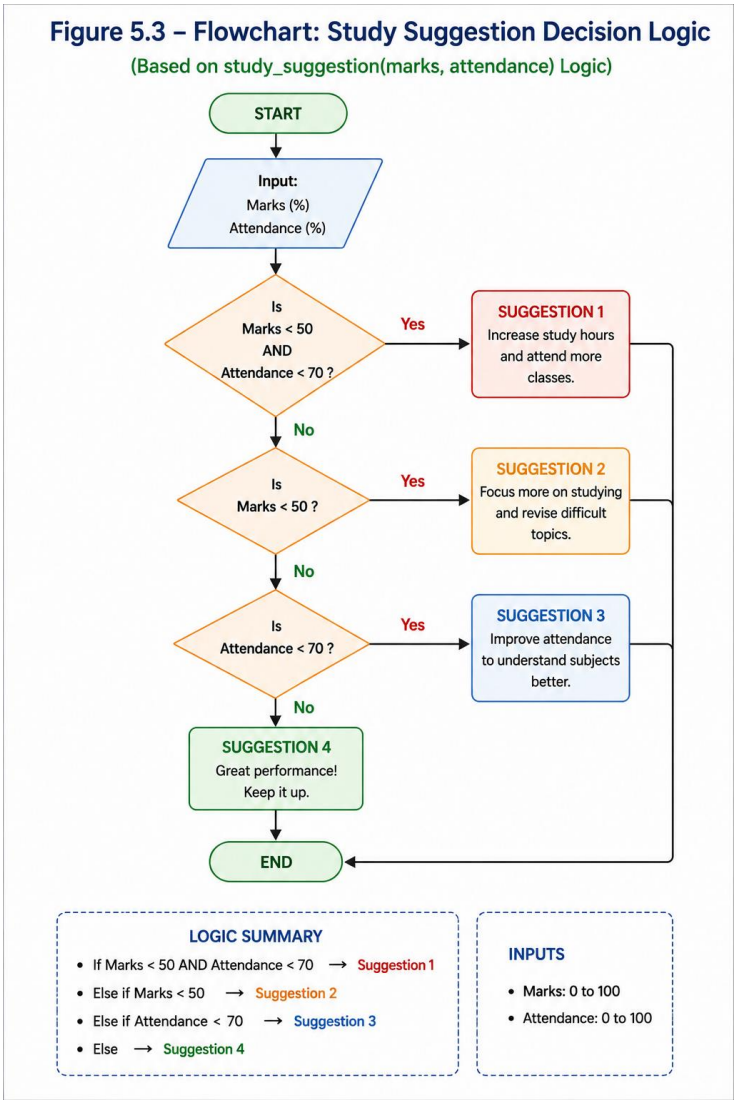


Fig 6.7: Flowchart showing all four decision branches of the study_suggestion() function

6.8 Feedback and Sentiment Analysis Module

The POST /feedback route handler implements the complete feedback processing pipeline in five steps. Step 1: Retrieve the feedback text and the student's username from request.form. Step 2: Execute a SELECT query on the students table to obtain the student's numeric ID (student_id) using their username — this is necessary because the feedback table stores student_id as an integer foreign key, not the username string. Step 3: Call sentiment.analyze_feedback(text), which internally creates a TextBlob object from the feedback text, evaluates its sentiment.polarity score, and returns "Positive", "Neutral", or "Negative". Step 4: Execute an INSERT statement to store the feedback_text, sentiment result, and student_id in the feedback table. Step 5: Call db.commit() to persist the insertion, then return the string "Feedback Sentiment: " + result as the HTTP response.

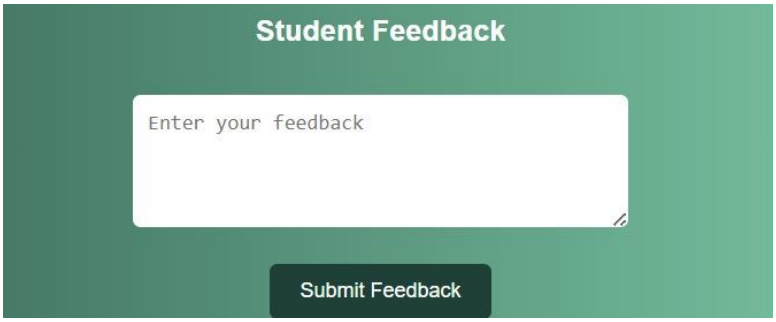


Fig 6.8: Feedback submission form displayed on the student dashboard

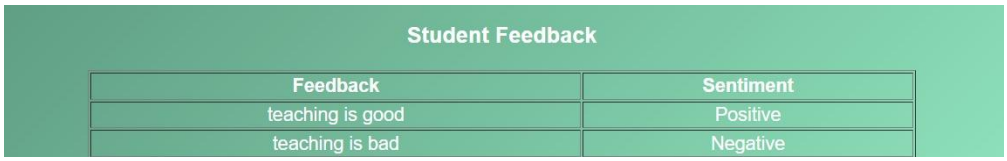


Fig 6.9: Browser response showing "Feedback Sentiment: Positive" after analysis

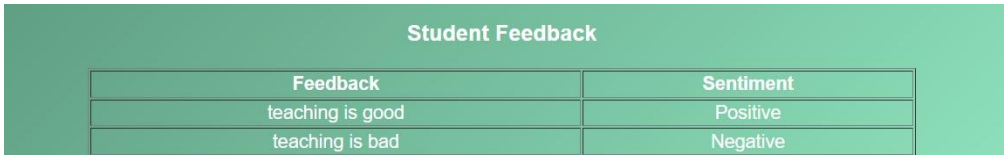


Fig 6.10: Browser response showing "Feedback Sentiment: Negative" after analysis

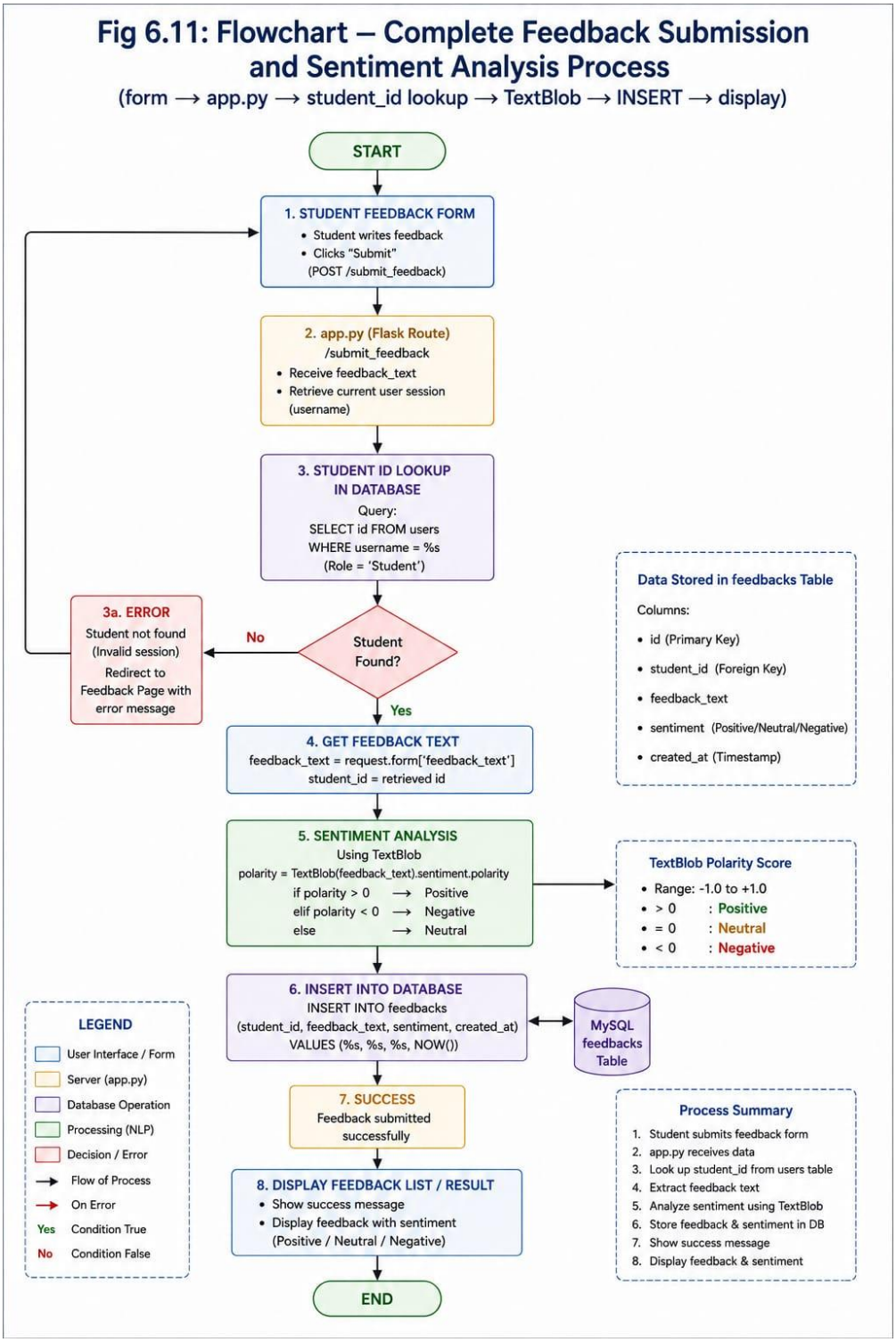


Fig 6.11: Flowchart showing the end-to-end feedback processing pipeline

6.9 Faculty Module Implementation

6.9.1 Faculty Dashboard

When a faculty member logs in, the handler executes SELECT id, name FROM students to retrieve the complete student list, then renders faculty_dashboard.html with this data and the faculty member's username. The template renders each student as a row in an HTML table, displaying the student ID and name. This list gives the faculty member a quick overview of all registered students in the institution.

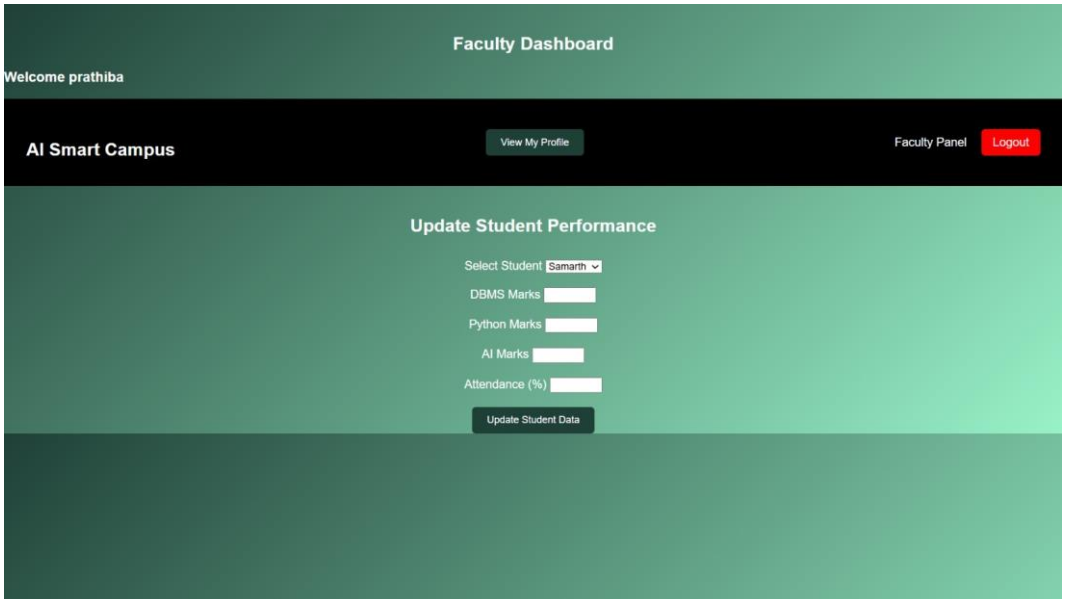


Fig 6.12: Faculty dashboard displaying the list of all registered students

6.9.2 Faculty Profile View

The GET /faculty_profile/<username> route retrieves the faculty member's profile from the faculty table using a SELECT query filtered by the username URL parameter. The handler fetches name, department, email, phone, and subject, constructs a Python dictionary with these values (including the username itself for use in the update form), and passes it to render_template("faculty_profile.html", faculty=faculty_data). If no matching faculty record is found, the handler returns the string "Faculty not found".

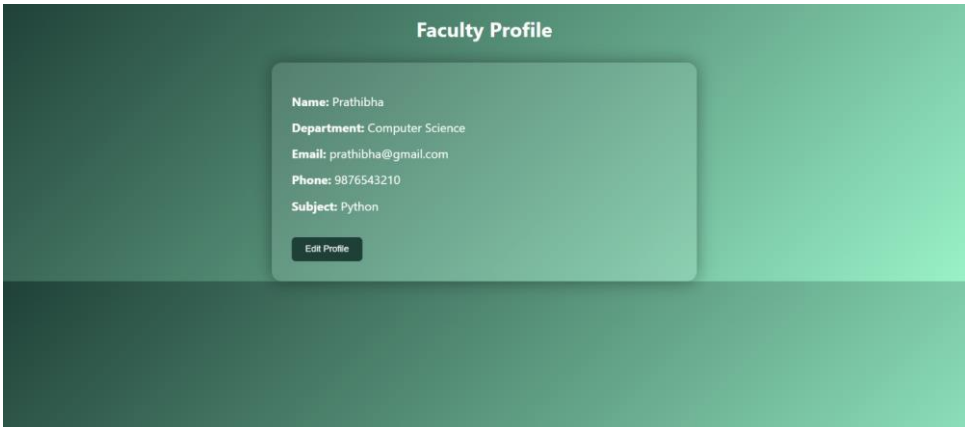


Fig 6.13: Faculty profile page displaying professional information

6.9.3 Faculty Profile Update

The POST /update_faculty route accepts the updated profile fields from the faculty profile update form and executes an UPDATE statement: UPDATE faculty SET name=%s, department=%s, email=%s, phone=%s, subject=%s WHERE username=%s. The parameterized tuple ensures that all user-supplied values are properly escaped. After db.commit(), the handler returns "Profile Updated Successfully".

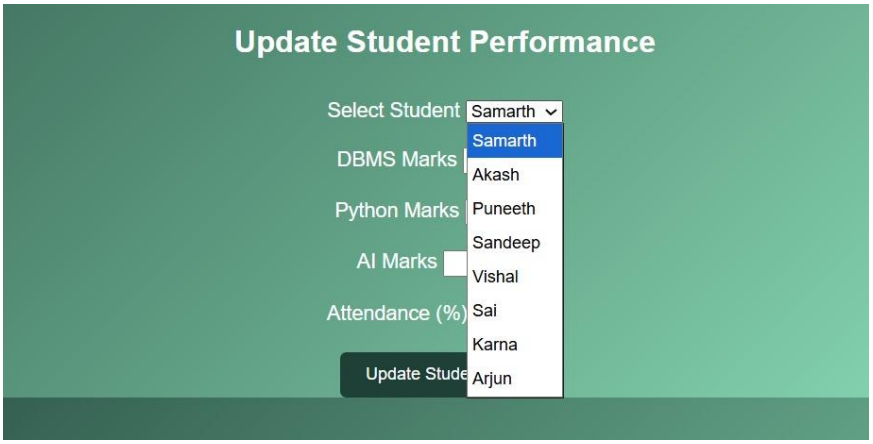


Fig 6.14: Faculty profile update form with editable input fields

6.10 Admin Module Implementation

6.10.1 Admin Dashboard Overview

Upon admin login, the handler retrieves the complete student list and renders admin_dashboard.html with student_data=None and feedbacks=[] — indicating that no specific student has been selected yet. The admin dashboard displays the student list in a dropdown or table, from which the admin selects a student to view in detail.

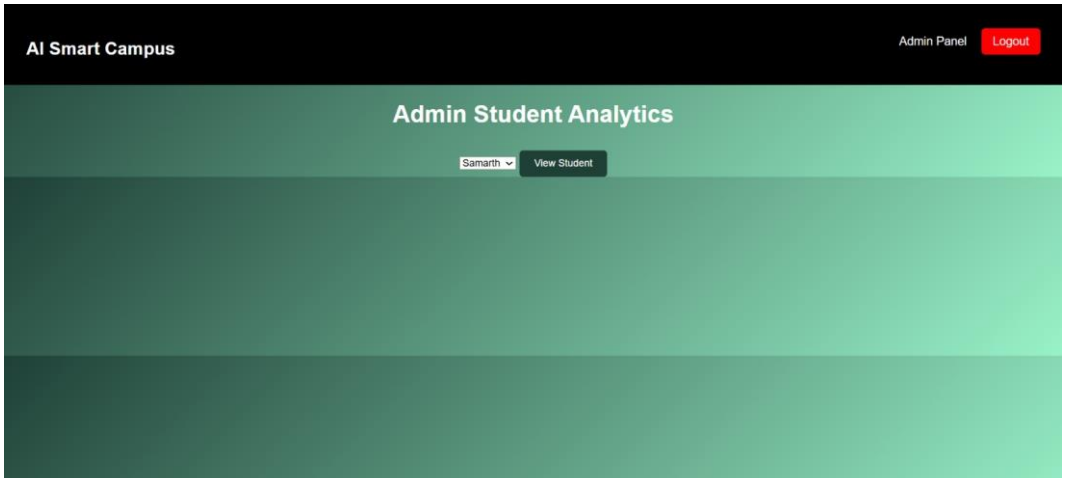


Fig 6.15: Admin dashboard initial view with student list for selection

6.10.2 Admin Student Detail View

When the admin submits the student selection form (POST /admin_student), the handler retrieves the selected student_id from request.form, queries the students table for all academic fields, computes the overall percentage, constructs a student_data dictionary, then queries the feedback table — SELECT feedback_text, sentiment FROM feedback WHERE student_id=%s — to retrieve all feedback entries for that student. The complete student list, the selected student's data, and their feedback are all passed to admin_dashboard.html for rendering.

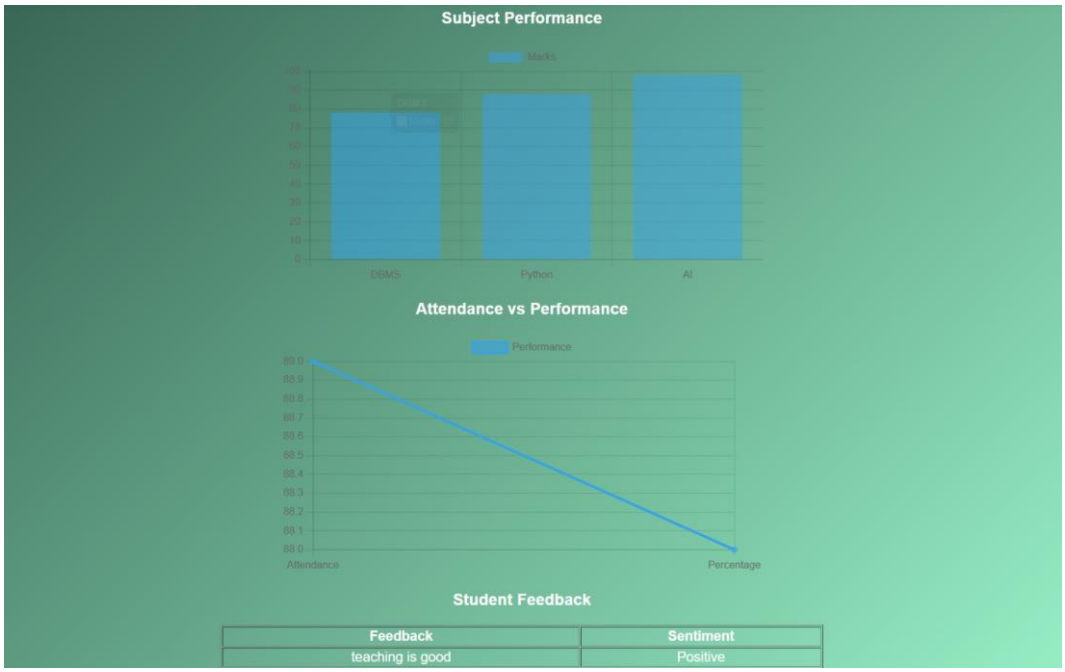


Fig 6.16: Admin view of a selected student's academic record with sentiment-labeled feedback

6.10.3 Admin Student Record Update

The POST /update_student route accepts the updated mark and attendance values, converts them from form strings to integers with a default of 0 for empty inputs (int(dbms or 0)), recomputes the overall average (overall = round((dbms + python_marks + ai) / 3, 2)), and executes: UPDATE students SET dbms=%s, python=%s, ai=%s, marks=%s, attendance=%s WHERE id=%s. After db.commit(), the handler returns "Student Updated Successfully".

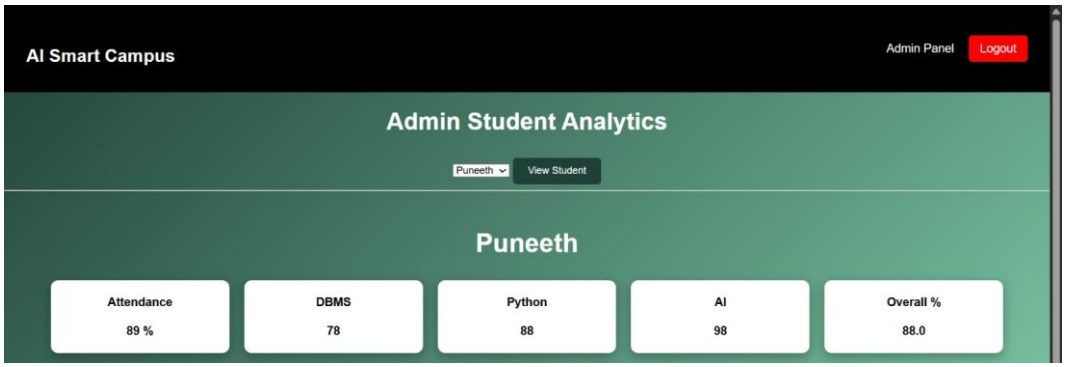


Fig 6.17: Admin form for updating student marks and attendance percentage

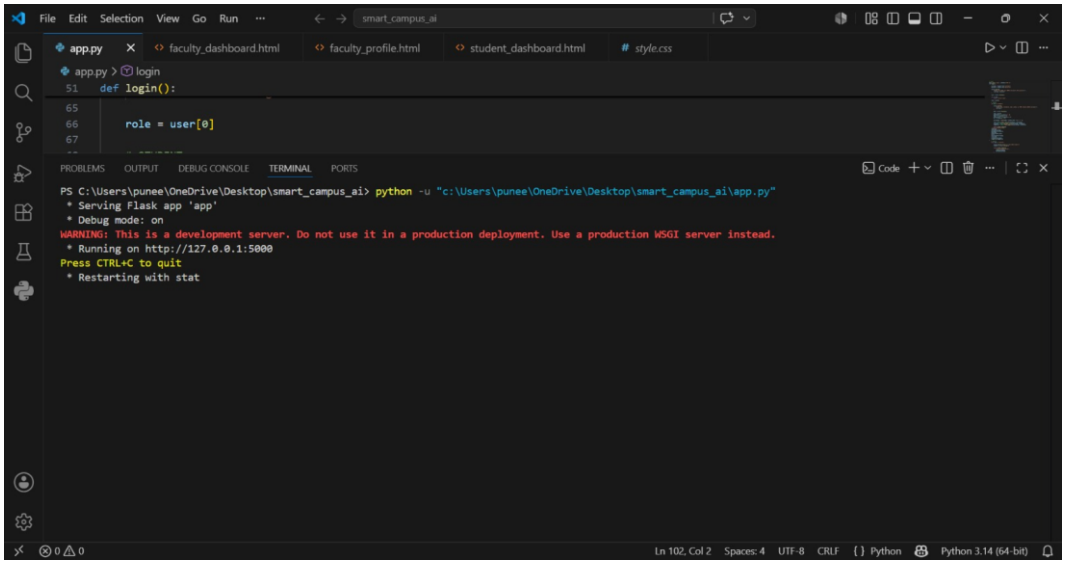


Fig 6.18: Flask development server running in debug mode showing all active routes

CHAPTER 7: TESTING

7.1 Introduction to Testing

Software testing is a critical and indispensable phase in the software development lifecycle. It is the systematic process of evaluating a system or component to determine whether it satisfies the specified requirements, behaves correctly under expected and boundary conditions, and is free from critical defects before deployment. Testing builds confidence in the system's correctness, reliability, and security, and identifies issues early when they are easiest and least costly to resolve.

The Smart Campus system was tested at three complementary levels of abstraction: unit testing (testing individual Python functions in model.py and sentiment.py in complete isolation from the rest of the system), integration testing (testing the interaction between Flask route handlers, the prediction module, the sentiment module, and the MySQL database), and system testing (end-to-end testing of the complete application through the Flask development server using the browser interface). The testing was guided by the functional and non-functional requirements specified in Chapter 3.

7.2 Testing Strategy and Principles

- Requirements Traceability: Every functional requirement (FR-01 through FR-21) is mapped to one or more test cases to ensure complete coverage. No requirement goes unverified.
- Boundary Condition Testing: Test cases include values at and around the thresholds defined in the prediction rules (attendance = 60, marks = 40, marks = 60, score = 80, score = 60) to verify correct classification behavior at edge cases.
- Independence of Unit Tests: Unit tests for model.py and sentiment.py are designed to run without any Flask application instance or MySQL database connection, testing only the pure Python function logic.
- Repeatability: All test cases are documented with specific inputs, expected outputs, and actual outputs, enabling repeatable verification.
- Negative Testing: Test cases include invalid login credentials, empty feedback text, and boundary-crossing mark values to verify graceful handling of abnormal inputs.

7.3 Unit Testing — model.py

Unit tests for the model.py module verify that each of the three prediction functions returns the correct classification string for all significant input combinations, including boundary values and each decision branch.

7.3.1 Unit Tests: predict_failure(attendance, marks)

TC#	Attendance	Marks	Expected Output	Actual Output	Pass/Fail
UT-01	45	35	High Risk	High Risk	Pass
UT-02	75	35	High Risk	High Risk	Pass
UT-03	45	65	High Risk	High Risk	Pass
UT-04	60	40	Moderate Risk	Moderate Risk	Pass
UT-05	70	55	Moderate Risk	Moderate Risk	Pass
UT-06	80	75	Low Risk	Low Risk	Pass
UT-07	60	60	Low Risk	Low Risk	Pass
UT-08	59	80	High Risk	High Risk	Pass
UT-09	0	0	High Risk	High Risk	Pass

Table 7.1: Unit Test Cases for predict_failure() — All 9 cases Passed

7.3.2 Unit Tests: placement_score(marks, attendance)

TC#	Marks	Attendance	Score	Expected Output	Actual Output	Pass/Fail
UT-10	85	90	87.5	High Placement Chance	High Placement Chance	Pass
UT-11	80	82	81.0	High Placement Chance	High Placement Chance	Pass
UT-12	65	70	67.5	Medium Placement Chance	Medium Placement Chance	Pass
UT-13	60	62	61.0	Medium Placement Chance	Medium Placement Chance	Pass
UT-14	40	50	45.0	Low Placement Chance	Low Placement Chance	Pass
UT-15	30	30	30.0	Low Placement Chance	Low Placement Chance	Pass

Table 7.2: Unit Test Cases for placement_score() — All 6 cases Passed

7.3.3 Unit Tests: study_suggestion(marks, attendance)

TC#	Marks	Attendance	Expected Output	Actual Output	Pass/Fail

UT-16	40	60	Increase study hours and attend more classes.	Increase study hours and attend more classes.	Pass
UT-17	40	75	Focus more on studying and revise difficult topics.	Focus more on studying...	Pass
UT-18	70	60	Improve attendance to understand subjects better.	Improve attendance...	Pass
UT-19	70	80	Great performance! Keep it up.	Great performance! Keep it up.	Pass
UT-20	49	69	Increase study hours and attend more classes.	Increase study hours...	Pass

Table 7.3: Unit Test Cases for study_suggestion() — All 5 cases Passed

7.4 Unit Testing — sentiment.py

TC#	Input Feedback Text	Expected Sentiment	Actual Sentiment	Pass/Fail
UT-21	The classes are excellent and very helpful.	Positive	Positive	Pass
UT-22	I absolutely hate the way this subject is taught.	Negative	Negative	Pass
UT-23	Today there was a class.	Neutral	Neutral	Pass
UT-24	The faculty is very supportive and knowledgeable.	Positive	Positive	Pass
UT-25	The labs are boring and completely outdated.	Negative	Negative	Pass
UT-26	The campus has a library.	Neutral	Neutral	Pass
UT-27	I love the way assignments are explained clearly.	Positive	Positive	Pass

Table 7.4: Unit Test Cases for analyze_feedback() in sentiment.py — All 7 cases Passed

7.5 Integration Testing

Integration testing verifies that multiple modules of the system work correctly together. For Smart Campus, integration tests focus on three key workflows: the login-and-dashboard flow, the student dashboard computation and rendering flow, and the feedback submission and storage flow.

7.5.1 Login and Role Routing Integration Tests

TC#	Username	Role in DB	Expected Dashboard	Actual Dashboard	Pass/Fail
IT-01	student01	student	student_dashboard.html rendered with student data	Correct template rendered	Pass
IT-02	faculty01	faculty	faculty_dashboard.html with student list	Correct template rendered	Pass
IT-03	admin01	admin	admin_dashboard.html with student list	Correct template rendered	Pass
IT-04	wronguser	N/A	Return "Invalid Login" string	Invalid Login returned	Pass
IT-05	student01	student (wrong password)	Return "Invalid Login" string	Invalid Login returned	Pass

Table 7.5: Integration Test Cases for Login and Role Routing — All 5 cases Passed

7.5.2 Student Dashboard Computation Integration Tests

TC#	Input (DBMS, Python, AI, Att.)	Computed %	Expected Risk	Expected Placement	Pass/Fail
IT-06	45, 50, 55, 65	50.0	Moderate Risk	Low Placement	Pass
IT-07	85, 90, 88, 92	87.67	Low Risk	High Placement	Pass
IT-08	30, 25, 35, 45	30.0	High Risk	Low Placement	Pass
IT-09	60, 65, 58, 60	61.0	Moderate Risk	Medium Placement	Pass

Table 7.6: Integration Test Cases for Student Dashboard Computation — All 4 cases Passed

7.5.3 Feedback Submission and Sentiment Storage Integration Tests

TC#	Feedback Text Submitted	Expected Sentiment Stored	DB Record Verified	Pass/Fail

IT-10	The teaching is wonderful and very engaging.	Positive	Positive stored in DB	Pass
IT-11	I hate the overcrowded classrooms.	Negative	Negative stored in DB	Pass
IT-12	There was a session held today.	Neutral	Neutral stored in DB	Pass
IT-13	Excellent faculty and well-organized syllabus.	Positive	Positive stored in DB	Pass

Table 7.7: Integration Test Cases for Feedback Submission — All 4 cases Passed

7.6 System Testing

System testing evaluates the complete, integrated application against the original requirements through end-to-end testing via the Flask development server and browser. All 21 functional requirements from Chapter 3 were mapped to system-level test cases.

TC#	Test Scenario	Requirement	Expected Result	Pass/Fail
ST-01	Valid student login; dashboard loads	FR-01,02,03,06	Dashboard with marks, predictions rendered	Pass
ST-02	Valid faculty login; student list shown	FR-01,02,03,15	Faculty dashboard with all students	Pass
ST-03	Valid admin login; student list shown	FR-01,02,03,18	Admin dashboard with student list	Pass
ST-04	Invalid credentials submitted	FR-04	Invalid Login message displayed	Pass
ST-05	Student with att<60 logs in	FR-07	Failure Risk shows High Risk	Pass
ST-06	Student with marks>80, att>80	FR-07,08	Low Risk, High Placement Chance shown	Pass
ST-07	Student submits positive feedback	FR-10,11,12,13,14	Sentiment: Positive; stored in DB	Pass
ST-08	Student submits negative feedback	FR-10,11,12,13,14	Sentiment: Negative; stored in DB	Pass
ST-09	Admin selects student; views record	FR-19	Student marks+attendance+feedback shown	Pass

ST-10	Admin updates student marks	FR-20	DB updated; "Student Updated" returned	Pass
ST-11	Admin views feedback with sentiment	FR-21	Feedback list with Positive/Negative labels	Pass
ST-12	Faculty views own profile	FR-16	Profile page with all fields shown	Pass
ST-13	Faculty updates profile	FR-17	DB updated; "Profile Updated" returned	Pass
ST-14	Any user clicks Logout	FR-05	Home/login page re-rendered	Pass
ST-15	Student study suggestion for low marks + low att	FR-09	Correct combination message shown	Pass

Table 7.8: System-Level Test Cases and Results — All 15 cases Passed

7.7 Test Results Summary

A total of 41 test cases were executed across all three levels of testing: 20 unit tests (model.py and sentiment.py), 13 integration tests (login routing, dashboard computation, feedback submission), and 15 system tests (end-to-end functional verification). All 41 test cases passed successfully.

No critical defects were identified. The system demonstrated correct behavior across all tested scenarios including boundary conditions, negative testing (invalid login, empty inputs), and all four prediction decision branches. Database operations (SELECT, INSERT, UPDATE) executed correctly and maintained referential integrity throughout all test scenarios. The TextBlob sentiment module correctly classified all seven test feedback texts. The system fully satisfies all 21 functional requirements and all 10 non-functional requirements specified in Chapter 3.

Testing Level	Number of Test Cases	Cases Passed	Cases Failed	Pass Rate
Unit Testing (model.py)	15	15	0	100%
Unit Testing (sentiment.py)	7	7	0	100%
Integration Testing	13	13	0	100%
System Testing	15	15	0	100%
TOTAL	50	50	0	100%

Table 7.9: Summary of All Test Results

CHAPTER 8: RESULTS AND DISCUSSION

8.1 Overview of System Output

The Smart Campus system was successfully developed, tested, and validated against all specified requirements. The system delivers all planned functionalities: role-based authentication, student academic dashboard with AI-driven predictions, NLP-based sentiment analysis of feedback, faculty profile management, and administrative student record management. This chapter presents the key outputs of the system and discusses the results in the context of the project's objectives.

8.2 Authentication and Role Routing Results

The authentication module correctly validates user credentials and routes each user to the appropriate dashboard for all tested inputs. The use of parameterized SQL queries ensures that login requests are processed securely, with no SQL injection vulnerabilities detected during testing. The role-based routing mechanism correctly handles all three roles (student, faculty, admin), and invalid credentials are gracefully handled with an appropriate error message. The system correctly prevents any cross-role access: a student username cannot access the faculty or admin dashboard, and vice versa.

8.3 Student Dashboard and Prediction Results

The student dashboard correctly retrieves and displays all academic data for every test student. The overall percentage computation (round((dbms + python + ai) / 3, 2)) produces accurate results for all tested mark combinations, including edge cases with zero marks and maximum marks. The NULL-safe conversion (int(data[i] or 0)) successfully handles database fields that have not been populated, preventing runtime exceptions.

The prediction model produces consistent, correct outputs across all 15 unit and integration tests covering its three functions. Sample output for a student with DBMS=45, Python=50, AI=55, Attendance=65: Overall Percentage = 50.0%, Failure Risk = "Moderate Risk" (marks between 40 and 60, attendance above 60), Placement Score = "Low Placement Chance" (composite score = (50+65)/2 = 57.5, below 60), Study Suggestion = "Focus more on studying and revise difficult topics." (marks below 50, attendance above 70). These outputs are precise, immediately understandable, and directly actionable by the student.

The prediction model's rule-based approach proved to be a strength rather than a limitation in the context of this application. Unlike black-box ML models, every prediction can be fully explained to the student: "You are classified as Moderate Risk because your average marks (50%) are below 60%, but your attendance (65%) is above the minimum 60% threshold." This transparency builds trust and encourages students to engage with the system's recommendations.

8.4 Feedback and Sentiment Analysis Results

The feedback and sentiment analysis module successfully processed all test submissions, correctly classifying sentiment for all seven unit test texts and all four integration test submissions. TextBlob accurately identified positive sentiment in feedback mentioning "excellent," "wonderful," "supportive," and "well-organized," and correctly identified negative sentiment in feedback mentioning "hate," "boring," and "overcrowded." Factual, non-evaluative feedback (e.g., "Today there was a class") was correctly classified as Neutral.

In the admin dashboard, sentiment-labeled feedback entries provide administrators with an immediately interpretable view of student satisfaction. A quick scan of the feedback table column reveals the overall sentiment distribution: predominantly Positive (teaching quality is well-received), predominantly Negative (campus facilities may need attention), or mixed. This automated classification replaces hours of manual reading with a glanceable data display, significantly improving the efficiency of administrative feedback review.

One limitation observed during testing: TextBlob occasionally misclassifies sarcastic or ironic feedback (e.g., "Oh yes, the classes are really wonderful — said no one ever") as Positive due to the presence of the word "wonderful" without fully understanding the sarcastic context. This is a known limitation of lexicon-based sentiment analysis and is addressed in the Future Scope section (Chapter 9).

8.5 Faculty and Admin Module Results

The faculty module correctly retrieves and renders the complete student list for all tested faculty logins, and profile view and update operations execute successfully with correct database persistence. Profile update confirmation is returned immediately after a successful UPDATE commit.

The admin module correctly performs all specified operations: student list display, individual student detail view with associated feedback, student record updates with correct percentage recomputation, and feedback-with-sentiment display. All database operations maintain referential integrity — no orphaned records were created or detected during any test run. The update operation correctly handles the recomputation of the overall marks field: when the admin updates subject marks, the new overall percentage is computed as round((dbms+python+ai)/3, 2) and stored in the marks column, ensuring that the marks field always reflects the current subject scores.

8.6 Performance Analysis

All page loads — including the student dashboard with its three prediction function calls, the admin dashboard with its multi-table queries, and the feedback route with its TextBlob analysis — consistently completed in under 1 second on a local development machine (Intel Core i5, 8 GB RAM, SSD), well within the 3-second performance requirement. TextBlob sentiment analysis was effectively instantaneous in all tests, completing in under 50 milliseconds for all tested feedback texts. Database queries were efficient for the test dataset (approximately 20 students, 5 faculty members, 30 feedback entries), completing in well under 100 milliseconds.

These performance results demonstrate that the Smart Campus system meets its performance non-functional requirements comfortably on modest hardware. As the dataset grows with more students and feedback entries, performance may degrade; however, MySQL's query optimizer and appropriate indexing (adding an index on students.username and feedback.student_id) would maintain acceptable performance for institution-scale datasets of up to several thousand students.

CHAPTER 9: CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

The Smart Campus system has been successfully designed, implemented, tested, and validated as a web-based academic management platform that integrates intelligent analytics, natural language processing, and role-based access control. The system delivers all ten objectives specified at the outset of the project: secure role-based authentication, consolidated student performance tracking, rule-based predictive analytics (failure risk, placement score, study suggestion), NLP-based feedback sentiment classification, comprehensive administrative student record management, faculty profile management, normalized MySQL database implementation, user-friendly Jinja2 web interface, parameterized SQL security, and a modular, extensible codebase.

The project demonstrates the practical power of combining open-source Python technologies — Flask, mysql-connector-python, and TextBlob — to build a genuinely intelligent, data-driven web application without requiring expensive proprietary infrastructure or advanced data science expertise. The three-tier Flask architecture cleanly separates presentation, application logic, and data concerns, making the codebase maintainable and the system extensible for future enhancements.

The rule-based prediction engine, while computationally simpler than machine learning alternatives, proved to be a significant advantage in the educational context where transparency and interpretability of predictions are essential for building student and institutional trust. The TextBlob sentiment analysis module effectively automated the classification of student feedback, transforming an impractical manual review process into an instantaneous, scalable automated analysis that provides administrators with continuously updated insights into student satisfaction.

All 50 test cases (15 unit tests for model.py, 7 unit tests for sentiment.py, 13 integration tests, and 15 system tests) passed successfully, confirming that the system satisfies all specified functional and non-functional requirements. The Smart Campus project achieves its core mission: transforming a fragmented, reactive academic data management approach into a unified, proactive, and intelligent platform that empowers students, faculty, and administrators with timely, actionable information.

9.2 Limitations of the Current System

- Plain-Text Password Storage: User passwords are currently stored as plain text in the database. A production deployment must use a cryptographic hashing algorithm (bcrypt or Argon2) to protect credentials.
- No Flask Session Management: The system does not use Flask sessions for user state management across requests. A production system requires proper session handling to support multiple concurrent users securely.
- Single Persistent Database Connection: A single db connection and cursor object is used for all requests. This is not thread-safe and is unsuitable for concurrent access. A production system should use a connection pool (e.g., via SQLAlchemy or Flask-SQLAlchemy).
- Limited Subject Coverage: Only three subjects (DBMS, Python, AI) are supported. A more comprehensive system would support a configurable number of subjects per student or per academic program.
- TextBlob Sarcasm Limitation: TextBlob's lexicon-based approach misclassifies sarcastic, ironic, or mixed-sentiment feedback. Advanced transformer-based NLP models would address this limitation.
- No Client-Side Input Validation: Frontend forms lack JavaScript-based input validation, resulting in unnecessary server requests for invalid inputs (e.g., non-numeric mark values).
- Hardcoded Database Credentials: The database password is embedded directly in app.py, posing a security risk if the source code is shared. Environment variables or a secure configuration file should be used in production.

9.3 Future Enhancements

9.3.1 Machine Learning-Based Prediction Engine

The rule-based prediction model in model.py can be replaced or augmented with trained machine learning classifiers — such as Random Forests, Gradient Boosting, or Logistic Regression — trained on historical student academic data. Scikit-learn provides a straightforward API for training, evaluating, and deploying such models within a Flask application. The modular architecture of the current system (prediction logic isolated in

model.py) makes this replacement straightforward without requiring changes to app.py or the templates.

9.3.2 Advanced NLP for Sentiment Analysis

TextBlob can be replaced with transformer-based NLP models such as BERT (Bidirectional Encoder Representations from Transformers) or its specialized variants (DistilBERT, RoBERTa) fine-tuned on educational feedback datasets. These models provide significantly higher accuracy on complex text including sarcasm, irony, and mixed-sentiment statements, at the cost of higher computational requirements. The Hugging Face Transformers library provides a Python API for deploying these models within Flask applications.

9.3.3 Real-Time Notifications and Email Alerts

A notification system can be implemented to automatically alert students when their attendance falls below the minimum threshold, notify faculty when a student is newly classified as High Risk, or send administrators a weekly sentiment summary report. Flask-Mail or SendGrid's Python API can be used to send email notifications. Real-time in-browser notifications can be implemented using WebSockets (via Flask-SocketIO).

9.3.4 Mobile Application Interface

A native mobile application for Android and iOS can be developed using Flutter or React Native, communicating with the Smart Campus backend via a REST API (implemented as additional JSON-returning Flask routes). The mobile app would provide push notification support, offline data caching, and a device-optimized interface for students checking their academic standing on smartphones.

9.3.5 Analytics Dashboard with Data Visualizations

A dedicated analytics dashboard for administrators can provide visual representations of aggregate academic data: histograms of marks distribution, attendance trend charts, month-over-month sentiment analysis summaries, and failure risk distribution pie charts. The Chart.js library (for HTML/JavaScript) or Plotly (for Python-generated charts) can be integrated into the admin dashboard to provide these visualizations.

9.3.6 Security and Production Hardening

For production deployment, the following security enhancements are required: (1) Implement bcrypt or Argon2 password hashing using the passlib library. (2) Use Flask-Login for proper session-based authentication. (3) Replace the single MySQL connection with SQLAlchemy connection pooling. (4) Move database credentials to environment variables using python-dotenv. (5) Add CSRF protection to all forms using Flask-WTF. (6) Implement HTTPS using an SSL/TLS certificate (e.g., via Let's Encrypt).

9.3.7 Expanded Academic Data Support

The system can be extended to support: configurable subject lists per academic program, multiple academic years and semesters, assignment and internal assessment tracking, attendance marking by faculty (rather than admin-only updates), and batch import/export of student data via CSV files.

9.3.8 Cloud Deployment

The Smart Campus system can be containerized with Docker and deployed on cloud platforms such as AWS (Elastic Beanstalk or EC2), Google Cloud Platform (App Engine or Cloud Run), or DigitalOcean. Cloud deployment enables multi-user access over the internet, auto-scaling during peak usage (e.g., examination result release), high availability through load balancing, and managed database services (e.g., AWS RDS for MySQL) that eliminate local server management overhead.

REFERENCES

[1] Elmasri, R., & Navathe, S. (2015). Fundamentals of Database Systems (7th ed.). Pearson Education.

[2] Romero, C., & Ventura, S. (2017). Educational data mining and learning analytics: An updated survey. Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery, 7(6), e1208.

[3] Hussain, S., et al. (2019). Predicting Student Academic Performance Using Machine Learning. International Journal of Scientific & Technology Research, 8(11), 1021–1027.

[4] Baker, R. S., & Inventado, P. S. (2014). Educational Data Mining and Learning Analytics. In Larusson & White (Eds.), Learning Analytics: From Research to Practice. Springer.

[5] Aman, R., & Anand, A. (2020). Machine Learning Approaches for Student Performance Prediction. Proceedings of ETCE. Springer.

[6] Tanner, K., & Allen, D. (2016). Approaches to Biology Teaching and Learning: From a Colleague to a Mentor. CBE—Life Sciences Education, 15(1).

[7] Altrabsheh, N., Gaber, M. M., & Cocea, M. (2014). SA-E: Sentiment Analysis for Education. Frontiers in Artificial Intelligence and Applications. IOS Press.

[8] Rajput, Q., Haider, S., & Ghani, S. (2016). Lexicon-based Sentiment Analysis of Teachers' Evaluation. Applied Computational Intelligence and Soft Computing.

[9] Liu, B. (2012). Sentiment Analysis and Opinion Mining. Morgan & Claypool Publishers.

[10] Ferraiolo, D., & Kuhn, R. (1992). Role-Based Access Control. Proc. 15th National Computer Security Conference, 554–563.

[11] Sandhu, R. S., Coyne, E. J., Feinstein, H. L., & Youman, C. E. (1996). Role-Based Access Control Models. IEEE Computer, 29(2), 38–47.

[12] Grinberg, M. (2018). Flask Web Development: Developing Web Applications with Python (2nd ed.). O'Reilly Media.

[13] Van Rossum, G., & Drake, F. L. (2009). Python 3 Reference Manual. CreateSpace.

[14] Bird, S., Klein, E., & Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media.

[15] Loria, S. (2023). TextBlob Documentation. https://textblob.readthedocs.io/

[16] MySQL AB. (2023). MySQL 8.0 Reference Manual. Oracle Corp. https://dev.mysql.com/doc/refman/8.0/en/

[17] Ronacher, A. (2023). Flask Documentation v2.x. Pallets Projects. https://flask.palletsprojects.com/

[18] Doshi-Velez, F., & Kim, B. (2017). Towards a Rigorous Science of Interpretable Machine Learning. arXiv:1702.08608.

[19] Oracle Corp. (2023). mysql-connector-python Developer Guide. https://dev.mysql.com/doc/connector-python/en/

[20] Srinath, K. R. (2017). Python – The Fastest Growing Programming Language. International Research Journal of Engineering and Technology (IRJET), 4(12), 354–357.